

Fraud Detection in Decentralized Finance using Graphical Embedding and GNN

A Project Report

Submitted in Partial Fulfilment of the Requirement for the Award of the Degree of

BACHELOR OF TECHNOLOGY

(INFORMATION TECHNOLOGY)



To

Dr.A.P.J.ABDUL KALAM KALAM TECHNICAL UNIVERSITY, LUCKNOW

By

Abhishek Mishra 2207360130004

Praveen Sharma 2307360139002

Rishabh Bharti 2307360139004

Vikas Pratap 2307360139006

Under the Supervision of

Dr. Tauseef Ahmad

(Assistant Professor)



Department of Information Technology

Rajkiya Engineering College Azamgarh-276201

May 2026

© Rajkiya Engineering College, Azamgarh – 276201, U.P. 2026

All Rights Reserved

PLAGIARISM REPORT

We declare that this written project report tested on Humanize AI. The plagiarism software DupliChecker showing less than 20% similarity index after excluding cited contents.

Abhishek Mishra
(2207360130004)

Praveen Sharma
(2307360139002)

Rishabh Bharti
(2307360139004)

Vikas Pratap
(2307360139006)



Department of Information Technology
Rajkiya Engineering College Azamgarh , UP India-276201

DECLARATION

We declare that this written submission represents our work and ideas in our own words along with others ideas or words which have adequately cited and referenced from original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the university / institution and can also evoke penal action from the sources which have thus not been properly cited or from whom paper permission has not been taken when needed.

Date: / / 2026

Place: Azamgarh

Abhishek Mishra
(2207360130004)

Praveen Sharma
(2307360139002)

Rishabh Bharti
(2307360139004)

Vikas Pratap
(2307360139006)



Department of Information Technology
Rajkiya Engineering college Azamgarh , UP India-276201

CERTIFICATE

This is to certify that this project report entitled “**Fraud detection in Decentralized Finance using Graphical Embedding and GNN.**” Submitted By Abhishek Mishra ,Rishabh Bharti, Praveen Sharma ,Vikas Pratap to the Rajkiya Engineering College Azamgarh affiliated to Dr. A.P.J. Abdul Kalam Technical University, for the award of the degree of Bachelor of Technology in Information Technology, is a project work carried out by us under the supervision of **Dr. Tauseef Ahmad**, Asst. Prof. Department of Information Technology, Rajkiya Engineering College, Azamgarh. To the best of my knowledge this work has not been submitted in part or full to any other university or institution for the award of any degree or diploma.

Dr. Tauseef Ahmad

(Supervisor & HOD ITD)

External Examiner

ACKNOWLEDGEMENT

It is my great pleasure to express my sincere gratitude to all those who have supported me throughout the completion of my B.Tech project. Without their encouragement, guidance, and valuable assistance, this work would not have been possible. First and foremost, I would like to express my deepest gratitude to my project advisor, **Dr. Tauseef Ahmad**, for his constant guidance, support, and encouragement throughout the project. His valuable suggestions, insightful feedback, and scientific approach have been instrumental in shaping this work. His dedication, patience, and willingness to help at every stage made it possible for me to complete this project successfully and on time. I would also like to extend my sincere thanks to **Prof. B.K. Tripathi**, Director, Rajkiya Engineering College, Azamgarh, for providing the necessary resources and a conducive environment for learning and research. I am equally grateful to the faculty members and staff of the Department of Information Technology for their cooperation and support during the course of this project. I would like to acknowledge the motivation and guidance received while working on the project titled “Fraud Detection in Decentralized Finance using Graphical Embedding and GNN.” I am also thankful to my friends and peers for their continuous support, valuable discussions, and encouragement throughout the project. Finally, I express my heartfelt gratitude to my parents for their unconditional love, constant support, and blessings, which have always been my source of strength and inspiration. This acknowledgement would remain incomplete without expressing my deep sense of gratitude to everyone who directly or indirectly contributed to the successful completion of this project.

Abhishek Mishra

(2207360130004)

Praveen Sharma

(2307360139002)

Rishabh Bharti

(2307360139004)

Vikas Pratap

(2307360139006)

ABSTRACT

The innovation of Decentralized Finance (DeFi) has proven to be groundbreaking within the financial sector due to the ability of enabling direct financial transactions between two parties without the need for intermediation from banks or financial institutions. Leveraging the potential of blockchain technology, DeFi presents benefits of transparency, access, and decentralization of finances. Nonetheless, the decentralized, permissionless, and pseudonymous structure of DeFi applications renders them susceptible to various forms of fraud such as money laundering, phishing, rug pull, ponzi schemes, flash loan exploitation, among others. The conventional fraud detection algorithms have been rendered ineffective in such scenarios as a result of the intricate nature of blockchain transactions, which are interconnected, dynamic, and complex. In addressing the aforementioned problem, this project suggests a novel fraud detection algorithm based on GNNs and transaction analysis via graphs. In the suggested fraud detection algorithm, each blockchain transaction can be seen as a graph, where wallets are the nodes while transactions represent the edges. With the help of graph embeddings and Graph Neural Networks, the proposed system can efficiently learn about transaction behavior patterns as well as dependency patterns in between wallet addresses. The model can analyze critical behavioral features like transaction frequency, volume of transactions, interaction behavior, connection behavior, and relational networks to detect any form of fraudulent activity. Moreover, the system also employs suspicious account identification through transaction frequency and behavior analysis as well as neighborhood relationship analysis.

The proposed system is expected to enhance the performance of traditional machine learning approaches towards fraud detection systems for decentralized environments.

LIST OF FIGURE

Figure No.	Caption	Page No.
Figure 1.1:	Centralized and Decentralized Transaction.....	2
Figure 1.2:	DeFi Fraud Loss Distribution.....	3
Figure 1.3:	Graph Neural Network (GNN).....	11
Figure 3.1:	Transformation of Tabular Blockchain Ledger Data into a Directed Heterogeneous Graph.....	21
Figure 3.2:	Workflow of Model Proposed.....	22
Figure 3.3:	Local Neighborhood Aggregation and Message Passing in GNN.....	23
Figure 4.1:	Distributed Model Training via Federated Learning Framework	33
Figure 5.1:	Graph of Transaction.....	36
Figure 5.2:	Total Transactions: Legitimate (0) vs Fraudulent (1)	37
Figure 5.3:	Model Comparison.....	39
Figure 5.4:	ROC Curves	40
Figure 5.5:	Precision Recall Curve	41
Figure 5.6:	Confusion Matrix	42
Figure 5.7:	Validation F1 per Epoch	42
Figure 5.8:	Convergence Graph.....	43
Figure 5.9:	t-SNE Visualization of GATv2 Graph Embeddings.....	44

LIST OF TABLES

Table no. No.	Caption	Page
Table 1:	Summary of Litration Review	16
Table 2:	Complete Technology Stack Overview	28
Table 3:	Comparison Model Performance on Defi Ethereum and Elliptic Dataset	38
Table 4:	Fraud Pattern Detected Results	44
Table 5:	Computational Performance Benchmarks	45
Table 6:	SHAP Features Importance	45

Table of Content

PLAGRISM REPORT	i
DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
List of Figures	vi
List of Table	vii
List of Abbreviation	x
Chapter 1: Introduction	1
1.1 Introduction	1
1.2 Decentralize Finance	2
1.3 Motivation	3
1.4 Background of the Study	4
1.4.1 The DeFi Ecosystem and Associated Security Risks	5
1.4.2 Role of Machine Learning in DeFi Fraud Detection	5
1.4.3 Graph-Based Modelling and Graph Embeddings	6
1.4.4 Application of Graph Neural Networks in DeFi Fraud Detection	7
1.4.5 Advantages, Challenges, and Scope of the Proposed Approach	8
1.5 Problem Statement	9
1.6 Objectives	9
1.7 Scope	9
1.8 Significance of the Study	10
1.9 Overview of Graph Neural Network and Graph Embedding	10
Chapter 2: Literature Review	12
2.1 Literature Study	12
2.1.1 Evolution of fraud Detection Methods	12
2.1.2 Graph-Based Learning and Graph Neural Networks	12
2.1.3 Advanced GNN Architectures and Optimization	13
2.1.4 Spatial- Temporal GNNs for Blockchain Anomaly Detection	14
2.1.5 Network Topology and Fraud Pattern Detection	14
2.1.6 Decentralize Finance (DeFi) and Emerging Fraud Challenges	15

2.1.7 Explainability, Regularity Compliance , and Research Gaps	15
2.2 Research Gap	17
Chapter 3:Methodology.....	19
3.1 Introduction.....	19
3.2 Data Collection	20
3.3 Data Verification and Exploratory Analysis.....	20
3.4 Model Overview	20
3.5 Conceptual Workflow of Proposed Model.....	21
3.6 Model Description	22
3.7 Mathematical Formulation With Description	23
3.7.1 Graph Representation.....	23
3.7.2 Node Feature Representation.....	23
3.7.3 GNN Message Passing.....	23
3.7.4 Final Node Embedding	24
3.7.5 Fraud Prediction Function.....	24
3.7.6 Loss Function.....	24
3.7.7 Evaluation Metrics	24
3.8 Pseudo Code.....	25
Chapter 4:Implementation.....	28
4.1 Overview of the Implementation	28
4.2 Development Environment and Toolchain	29
4.2.1 Visual Studio Code	29
4.2.2 Google Colab	29
4.2.3 Version Control and CI/CD.....	30
4.3 Data Acquisition and Preprocessing Pipeline	30
4.3.1 Blockchain and Data Sources	30
4.3.2 Preprocessing and Feature Engineering.....	31
4.4 Graph Construction and Representation	31
4.5 GNN Model Architecture and Training	32
4.5.1 Implementation Architecture	32
4.5.2 Training Configuration.....	32
4.5.3 Federated Learning Implementation	33
4.6 Backend Implementation	33
4.7 Frontend Implementation.....	34

4.8 Deployment Architecture	35
Chapter 5: Results and Discussion	36
5.1 Experimental Result and Performance Evaluation	36
5.1.1 Graph of Transactions	36
5.1.2 Distribution of Classes and Imbalance in the Dataset	37
5.1.3 Model Performance Comparison	37
5.1.4 Comparative Analysis.....	38
5.1.5 Receiver Operating Characteristic (ROC) Analysis	39
5.1.6 Precision-Recall Performance Analysis.....	40
5.1.7 Confusion Matrix	41
5.1.8 Validation F1 Scores Evolution.....	42
5.1.9 Covergence Graph	43
5.1.10 t-SNE Graph Embeddings Visualization	43
5.1.11 Fraud Pattern Type Detection Results	44
5.1.12 Scalability and Inference Performance	45
5.1.13 SHAP Feature Importance Analysis	45
5.2 Testing and Quality Assurance.....	46
5.3 Chapter Summary	47
Chapter 6:Conclusion and Future Work	48
6.1 Conclusion	48
6.2 Limitation and Future Work.....	48
References	49

List of Abbreviation

Abbreviation	Full Form
AGG	Aggregation Function
AML	Anti-Money Laundering
ANN	Artificial Neural Network
SMOTEENN	Synthetic Minority Over-sampling Technique with Edited Nearest Neighbors
AUC	Area Under the Curve
CFMM	Constant Function Market Maker
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
DeFi	Decentralized Finance
DGL	Deep Graph Library
ERC	Ethereum Request for Comment
FL	Federated Learning
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
XAI	Explainable AI
JA-GNN	Jump-Attentive Graph Neural Network
JWT	JSON Web Token
KNN	k-Nearest Neighbors
MLP	Multi-Layer Perceptron
ROC	Receiver Operating Characteristic
SHAP	Shapley Additive Explanations
STGNN	Spatial-Temporal Graph Neural Network
TCN	Temporal Convolutional Network
TGN	Temporal Graph Network

Chapter 1: Introduction

1.1 Introduction

Rapid changes in digital technology have caused a fundamental change in the traditional finance model by converting it from a centralized to an open and decentralized financial system. In previous times, financial systems were operated by centralized institutions like banks and governments that validated and monitored all transactions. On the other hand, decentralization came up as a way to overcome the drawbacks of traditional financial systems such as transparency, high transaction costs, less accessibility, and reliance on intermediaries. The concept of decentralization gained tremendous importance when blockchain was introduced in 2008, providing a platform where transactions could be made in an untrusted manner without the involvement of any centralized institution. This technology gave rise to the idea of a new form of finance called Decentralized Finance (De-Fi). De-Fi is considered a modern finance system built using blockchain technology in which users can enjoy financial facilities in a peer-to-peer manner without any intermediaries or with the help of smart contracts.

Nevertheless, the distributed and permissionless architecture of DeFi presents several security issues. Firstly, without any centralized management, there is no entity accountable for supervising and stopping any acts of fraud. Moreover, participants in DeFi environments use fake identities, and, thus, they cannot be traced easily. The blockchain-based network that serves as the infrastructure of DeFi applications is quite complicated and dynamically developing; therefore, it is impossible to spot any irregular behavior with traditional approaches. Accordingly, different kinds of scams like phishing, rug pulls, unlawful transactions, money laundering, and exploitation of smart contracts' loopholes are prevalent in DeFi environments.

Moreover, the fast-paced development and deployment of the DeFi space have led to even higher risks in this regard, as numerous protocols and platforms are being created rapidly without undergoing proper security checks. The emergence of more complex fraudulent schemes is expected, which will be increasingly hard to uncover and combat. It should be noted that the implementation of effective security strategies is crucial not only from the asset protection perspective but also for preserving overall stability within the DeFi space.

1.2 Decentralize Finance

DeFi is a modern term for a financial system that uses blockchain technologies, allowing its participants to engage in various financial activities without any traditional middlemen such as commercial banks or brokers. In contrast to conventional financial systems, DeFi does not require any intermediary; all transactions are made within the blockchain using smart contracts.

The idea of DeFi was born out of the development of blockchain technologies, especially when the use of smart contracts on the Ethereum platform began. Smart contracts allow automating such financial activities as loans, investments, trading, etc. This is what makes DeFi attractive and popular nowadays. One of the main benefits of DeFi is high transparency, as all transactions are recorded on the chain's public ledger and can be checked by any user. Moreover, transactions are processed faster and cheaper than in traditional financial systems. Users have full control over their assets, eliminating the need to trust third parties shown in figure 1.1.

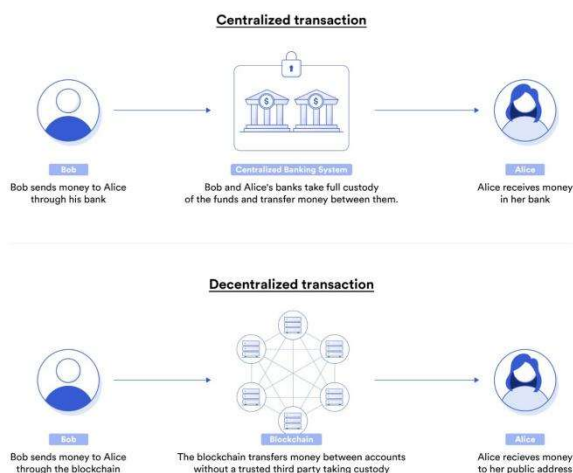


Figure 1.1: Centralized and Decentralized Transaction

Nevertheless, there are several problems that decentralized finance faces despite its benefits. The absence of any central authority hinders the ability to track and stop fraudulent operations. The anonymous nature of users and the intricacies of the blockchain technology make it even easier for criminals to perform such illegal actions as phishing, rug pulling, Ponzi scheme implementation, and money laundering.

1.3 Motivation

The rapid rise of Decentralized Finance (DeFi) has ushered in an era of change in the field of finance, changing the dynamics of financial transactions, their validation, and recording. While decentralization, anonymity, and the lack of regulation may be characteristics that make DeFi appealing and accessible, they may also offer opportunities for sophisticated and covert frauds. The decentralization, anonymity, and non-regulatory aspects of DeFi are all traits that have helped create an uncontrolled situation regarding transaction integrity and security.

The main motive behind selecting this research topic is that there is an increasing need to address fraud risks in the decentralized finance ecosystem in a more efficient way than the current approach. It is important to note that most fraud risk management mechanisms are designed for centralized systems and may not be efficient when handling intricate relationships in blockchain transaction networks.

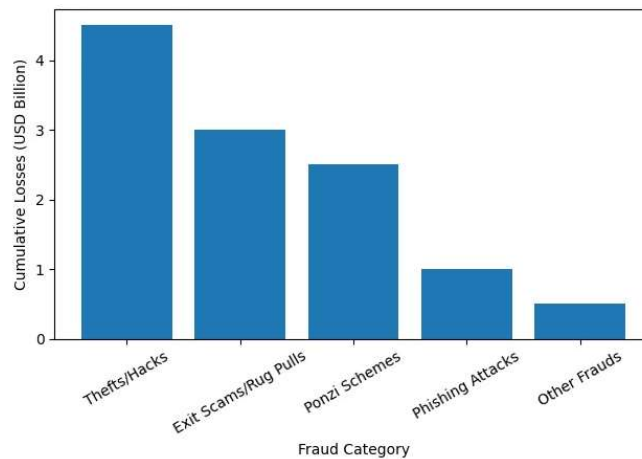


Figure 1.2: DeFi Fraud Loss Distribution

Moreover, the development of DeFi systems and the emergence of new financial instruments and protocols have escalated the degree of complexity of cyber risks and different type of threat shown in figure 1.2. In this regard, criminals utilize the intrinsic complexity and anonymity of DeFi systems to conduct criminal activities, including collusion, smart contract manipulation, and nested transactions. It is clear that there is a demand for sophisticated models capable of discovering any suspicious activity or patterns within extensive and dynamically changing datasets.

In light of this situation, the application of Graph Neural Networks and graph embeddings becomes a prospective approach to addressing the current problem. The fundamental advantage of graph-based techniques is their capacity to analyze relations within datasets and detect dependencies and patterns that cannot be revealed using traditional algorithms. Consequently, the main objective of this research is to identify ways in which these cutting-edge methods can improve fraud detection within DeFi platforms.

Ultimately, the present study aims to develop a robust framework for ensuring the security of decentralized financial systems, reducing risks, protecting users' assets, and enhancing trust in blockchain-based solutions.

1.4 Background of the Study

Blockchain technology provides the foundational infrastructure for DeFi systems, enabling the recording of all financial transactions in an immutable, transparent, and distributed manner. Each transaction is cryptographically validated by a network of independent nodes and appended to a chain of blocks, forming an ever-growing ledger that is practically tamper-proof. While this architecture ensures high transparency and security, it simultaneously generates enormous volumes of interconnected data whose complexity exceeds the analytical capabilities of conventional tools. Understanding these data characteristics is essential for designing effective fraud detection mechanisms.

Conventional fraud detection systems were developed specifically for centralized financial environments. They rely on predefined rule sets and relatively simple machine learning models that evaluate each transaction in isolation. In a DeFi system, however, every transaction is interlinked with others in complex and dynamic ways, forming transaction networks with intricate behavioral patterns. These interdependencies make it insufficient to analyse individual transactions without considering their relational context, demanding the development of more advanced analytical approaches.

1.4.1 The DeFi Ecosystem and Associated Security Risks

Decentralized Finance (DeFi) marks a transformative shift in the global financial system by leveraging blockchain technology to eliminate traditional intermediaries such as banks, brokers, and clearing houses. Built upon distributed ledger systems and smart contracts, DeFi platforms allow users worldwide to participate in financial activities including lending, borrowing, trading, and investing in a fully decentralized, trustless, and transparent manner. This innovation significantly lowers operational costs, increases financial accessibility for the unbanked population, and enables borderless peer-to-peer transactions at unprecedented speed and efficiency.

DeFi applications operate through decentralized applications (dApps) on blockchain networks such as Ethereum, executing logic via smart contracts with pre-programmed, self-enforcing rules. While the elimination of intermediaries grants users full autonomy over their assets, it simultaneously removes the accountability layer that centralised financial systems rely upon. Smart contracts, though powerful, frequently harbour vulnerabilities that malicious actors exploit. Moreover, the permissionless and open-source design of DeFi means that anyone can participate without undergoing identity verification, enabling the creation of multiple pseudonymous accounts and severely hampering traceability. These characteristics collectively create a fertile environment for various forms of financial fraud, including rug pulls, flash loan attacks, phishing scams, wash trading, Sybil attacks, Ponzi scheme implementations, and sophisticated money laundering operations. The rapid growth in DeFi adoption has correspondingly amplified the volume and complexity of these attacks, with cumulative losses from DeFi-related fraud surpassing twelve billion US dollars by 2023, making fraud detection an urgent and critical area of research.

1.4.2 Role of Machine Learning in DeFi Fraud Detection

Machine learning has brought about a paradigm shift in the domain of fraud detection, enabling systems to learn from historical data and autonomously identify complex behavioural patterns that deviate from normal activity. Unlike traditional rule-based systems that require manual definition of fraud indicators, machine learning models can generalise across unseen patterns and evolve with emerging fraud typologies. In the context of DeFi, classical machine learning

algorithms such as Logistic Regression, Support Vector Machines, Random Forests, and XGBoost have demonstrated promising results in identifying suspicious transactions based on extracted features. However, these models process transactions as independent, isolated entities and are therefore fundamentally limited in their ability to capture the multi-hop relational dependencies and coordinated behaviours that characterise the most sophisticated forms of DeFi fraud.

The structural inadequacy of conventional machine learning algorithms in handling graph-structured data represents a critical research gap. DeFi transaction networks are inherently relational — wallets interact with each other repeatedly, smart contracts aggregate and redistribute funds across clusters of addresses, and fraud rings operate through coordinated multi-party schemes. Capturing these relational properties requires a modelling paradigm that explicitly incorporates the graph topology of the transaction network, which is precisely the motivation for adopting Graph Neural Networks (GNNs) in this project.

1.4.3 Graph-Based Modelling and Graph Embeddings

One of the most effective approaches for addressing the relational complexity of DeFi transaction data is to model it as a graph. In this paradigm, each node in the graph represents a distinct entity — such as a user wallet, a smart contract address, or a decentralised exchange protocol — while each directed edge between two nodes represents a transaction or financial interaction between those entities, annotated with attributes such as transaction amount, timestamp, gas fee, and interaction type. This graph representation naturally captures the dependencies, asset flow pathways, and interaction patterns inherent in blockchain activity, enabling the detection of subtle structural anomalies that would be invisible in a flat, tabular data representation.

Graph embedding techniques play a fundamental and complementary role in this framework. These techniques transform nodes, edges, or entire subgraph structures into compact, low-dimensional vector representations while preserving the essential structural and relational properties of the original graph. By encoding information about a node’s position within the network, its neighbourhood connectivity, and its interaction patterns, graph embeddings create rich and computationally efficient feature representations that serve as inputs to downstream

machine learning models. Algorithms such as DeepWalk, Node2Vec, and LINE are widely used to generate such embeddings in an unsupervised manner, capturing both local and global structural characteristics of the transaction graph. The resulting embeddings enable machine learning classifiers to reason about entities based not only on their individual transaction features but also on their structural role and relational context within the broader transaction network.

1.4.4 Application of Graph Neural Networks (GNNs) in DeFi Fraud Detection

Graph Neural Networks (GNNs) represent a powerful class of deep learning architectures specifically designed to operate directly on graph-structured data. Unlike conventional neural networks that assume fixed-size, grid-like input formats (such as images or sequences), GNNs natively handle irregular and highly interconnected data structures. The fundamental operating principle of GNNs is the message-passing mechanism, through which each node iteratively aggregates feature information from its immediate neighbours, updates its own representation, and propagates enriched information further into the network across successive layers. This enables GNNs to capture not only the local structural properties around each node but also global relational patterns that emerge from the broader network topology. In the context of DeFi transaction graphs, GNNs are particularly effective because fraudulent activities frequently involve coordinated interactions among multiple entities: groups of malicious wallets may engage in circular transactions to obscure the origin of funds, exploit flash loan mechanisms through multi-step atomic operations, or form Sybil clusters through networks of synthetically generated identities. Such coordinated patterns are structurally distinctive in the transaction graph and can be reliably identified through the relational learning capabilities of GNNs.

Several specialised GNN architectures have demonstrated notable success in fraud detection tasks. Graph Convolutional Networks (GCNs) apply a spectral convolution operation over the graph to aggregate neighbourhood information, effectively learning node embeddings that reflect both node-level features and graph structure. Graph Attention Networks (GATs) extend GCNs by introducing a multi-head self-attention mechanism that assigns adaptive importance weights to different neighbouring nodes, allowing the model to focus on the most informative relationships within the transaction graph. GraphSAGE introduces inductive learning through neighbourhood sampling and aggregation, enabling efficient inference on previously unseen

nodes and facilitating scalable real-time fraud detection in dynamic, growing transaction graphs. The integration of graph embeddings with these GNN architectures yields a synergistic benefit: while graph embeddings simplify and enrich the input feature space, GNNs leverage these representations to perform deep relational reasoning, resulting in significantly improved classification accuracy and generalisability.

1.4.5 Advantages, Challenges, and Scope of the Proposed Approach

The adoption of GNNs and graph embedding techniques for DeFi fraud detection offers several compelling advantages over traditional approaches. First, GNN-based models achieve significantly higher accuracy by leveraging structural and relational features that rule-based and classical machine learning models entirely overlook. Second, GNNs demonstrate strong scalability through techniques such as mini-batch training and neighbourhood sampling, making them viable for deployment on large-scale, real-world DeFi transaction datasets comprising millions of wallets and transactions. Third, the flexibility of the GNN framework allows it to generalise and adapt to new fraud patterns over time, as the model can be fine-tuned on newly observed fraudulent behaviours without requiring a complete redesign. Fourth, the incorporation of graph embeddings into the pipeline improves computational efficiency by reducing the dimensionality of the feature space while retaining maximum informational content. Fifth, GNNs are naturally suited to detecting both individual fraudulent actors and organised fraud rings, as the message-passing mechanism allows the model to reason across multiple hops in the transaction graph.

Nevertheless, the application of GNNs to DeFi fraud detection is not without challenges. The high computational expense of training deep GNN models on large graphs is a significant practical constraint, particularly when real-time inference is required for live transaction scoring. Obtaining sufficient quantities of high-quality, ground-truth labelled fraud data is another persistent challenge, as confirmed fraud instances in decentralised environments are often scarce, partially observable, or subject to label noise. Furthermore, the class imbalance inherent in fraud datasets — where fraudulent transactions constitute a small minority of total activity — necessitates the application of specialised training strategies such as weighted loss functions, oversampling, and cost-sensitive learning to prevent the model from being biased towards the

majority class. The adversarial adaptability of fraudsters is also a concern: as detection systems improve, fraudsters modify their strategies to evade detection, requiring fraud detection models to be periodically retrained or designed with concept-drift awareness. Despite these challenges, the combination of graph embedding and GNN techniques represents the most promising and academically grounded approach to achieving robust, scalable, and interpretable fraud detection in decentralised finance ecosystems, forming the core technological motivation for this research project.

1.5 Problem Statement

Detecting fraud in DeFi systems is an interesting yet complicated problem owing to various reasons. Firstly, the absence of centralized authorities who would regulate transactions and monitor them is one challenge. The second problem lies in the very nature of user anonymity – since users' identities remain pseudonymous, it becomes more challenging to trace and recognize the identity of the person committing the crime. Another important difficulty related to DeFi systems refers to their complex transaction networks, in which many operations occur constantly.

1.6 Objectives

The main goal of the study presented in the paper is to develop an effective fraud detection approach based on graphs for decentralized finance platforms. Specifically, the purpose of the research involves investigating the use of GNNs for transaction network analysis and detection of potentially suspicious behavior. In addition, another key aspect under consideration involves employing approaches for graph representation to convert complex graph data into a form suitable for further classification. Moreover, in this case, it would also be possible to detect both individual and coordinated fraudulent actions by integrating the features and structure of the network.

1.7 Scope

The main goal of this research project is to detect any malicious activities in Decentralized Finance (DeFi). To achieve this aim, it will be crucial to model blockchain transactions in the form of graphs and analyze them using machine learning algorithms such as Graph Neural

Networks (GNNs). In this technique, transactions are transformed into graphs, where the nodes denote the entities such as the address of the wallet and edges represent the interactions between these entities.

In this project, the focus will be on developing a fraud detection model for DeFi by implementing machine learning algorithms such as GNNs and graph embedding. The graph embeddings transform the high dimensional graph to vector representations, and these vectors are further processed using GNN models to determine whether the transaction or entity is a fraud or not. Additionally, federated learning will be used to conduct simulations for decentralized model training, where each client trains its model locally and only shares the parameters of the model rather than the raw data.

Moreover, the experiments will be performed based on public Ethereum transaction datasets.

1.8 Significance of the Study

The importance of the study is that it can help improve the security and efficiency of the DeFi systems by introducing more sophisticated methods of detecting fraud. This way, not only will it help minimize the loss of assets but it will also build trust among users. The utilization of Graph Neural Networks and graph embeddings proves to be a more efficient technique of analyzing transaction networks than the one currently in use.

Furthermore, the current study also makes a contribution to the field of financial technology as it showcases the viability of using deep learning algorithms on graph data to solve practical problems.

1.9 Overview of Graph Neural Networks and Graph Embedding

Graph Neural Networks (GNNs) are neural networks tailored for analyzing graph data. The key principle of GNNs is that the nodes of a graph can communicate with their neighbors, which helps to discover patterns at the local and global level. During the learning process, the model represents nodes based on their position and role within the graph structure.

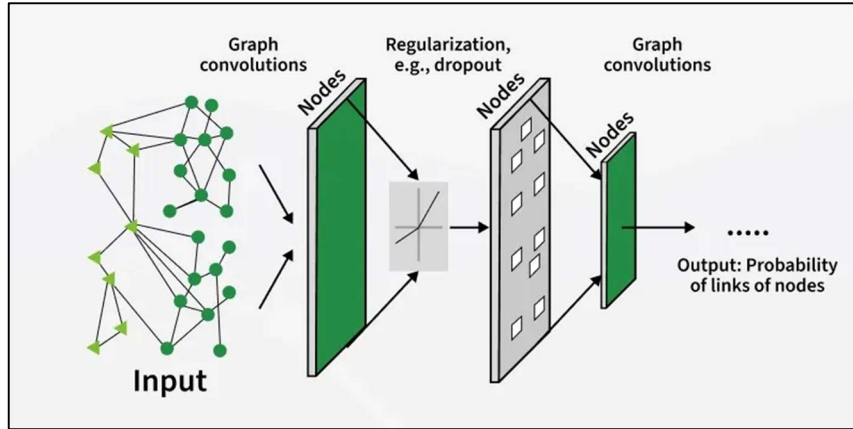


Figure 1.3: Graph Neural Network(GNN)

Graph embeddings are another set of methods used together with GNNs to represent nodes with vectors, preserving crucial information about their structure shown in figure 1.3. Graph embeddings can be utilized as features in machine learning algorithms, facilitating the analysis of massive graphs. Both graph embedding and GNN approaches can be employed to detect fraud patterns that involve collaboration among various entities.

Chapter 2: Literature Review

2.1 Literature Study

For the application of artificial intelligence in DeFi, the use of highly sophisticated GNN models has seen an exponential rise compared to traditional tabular-based models. This literature review focuses on various research work carried out recently regarding the detection of anomalies for fraud detection in blockchain technology through GNNs, based on seven important research papers, ranging from simple rule-based approaches to more advanced spatial-temporal and explainable graph learning approaches.

2.1.1 Evolution of Fraud Detection Methods

Detection of fraud was traditionally based on applying rule-based models and human intervention during the initial stages of recognizing suspicious activity through domain knowledge and past cases of fraud. Nevertheless, advanced patterns of fraud require greater flexibility, which cannot be provided by the use of static rules. Recently, there has been increased usage of machine learning algorithms to identify fraud. Such algorithms as logistic regression, support vector machines (SVM), K-nearest neighbors (KNN), random forest, and XGBoost have proven to be valuable tools in finance. According to [7], although the superior performance of these algorithms compared to traditional rule-based models in terms of analyzing transactional data and establishing non-linear connections is undeniable, it is not able to discover complicated dependencies between transactions and fraud patterns. Modern machine learning algorithms such as artificial neural networks (ANNs) and autoencoders have enabled feature extraction; however, as noted in [5], these models cannot recognize dependencies between financial transactions.

2.1.2 Graph-Based Learning and Graph Neural Networks

It is undeniable that the latest advances in graph representation learning methods have proven extremely useful for detecting fraud because of their capability to capture the relational structures of graphs. The graph representation learning methods discussed in the paper [9] modeled the financial transaction network using an example of a graph, $G = (V, E, X)$, wherein

V is the set of financial entities, E the transaction edges, and X node attributes, including transaction frequency and account age.

The GCN algorithm incorporates the structure of the graph and the characteristics of nodes concerning a neighborhood aggregation mechanism, where each node has its own embedding update by considering the information aggregated from neighboring nodes through the propagation function based on a normalized adjacency matrix, a diagonal degree matrix, and a feature matrix of each layer. The GCN network has shown impressive outcomes in detecting financial fraud because it performs better in learning intricate patterns in financial transaction graphs compared to traditional machine learning algorithms. In contrast, the GAT model improves the effectiveness of GCN by allocating adaptive weights to neighboring nodes to consider only the most significant nodes, ignoring the others, thereby becoming very effective in finding potential transaction relationships that lead to financial fraud. According to the paper [23], a technique for detecting financial fraud using dynamic directed graphs generated through rolling windows of five minutes and TGN that generates node embeddings with structural and transactional properties with the help of unsupervised Isolation Forest and LOF scoring methods achieved an excellent F1-score of 0.905 and AUC of 0.94, processing approximately 770 transactions per second.

2.1.3 Advanced GNN Architectures and Optimization Strategies

In order to identify fraudulent entities that camouflage themselves within genuine transactions by blending with authentic transaction clusters, the authors of the paper [7] suggest the JA-GNN algorithm that employs the Mahalanobis distance along with WMNC metric to unearth such fraudsters. The architecture of the suggested model includes attention and jump connection that keeps distant node features while avoiding the issue of over-smoothness associated with deep GNN architectures, and the algorithm was tested using Proprietary Financial Fraud dataset of 1.25 million observations as well as Yelp-Fraud benchmark dataset. Additionally, the paper [8] describes a hybrid artificial intelligence model that combines the use of GCN and GAT approaches along with explainable stacking ensemble, K-SMOTEENN, and Node2Vec techniques in identifying different fraud types unique to DeFi platforms such as flash loans and rug pulls on the Ethereum platform demonstrating the highest performance values of $F1 = 0.98$ and $AUC-ROC = 0.99$, respectively. Concerning scalability, the paper [4] reveals how the use of GraphSAGE can solve the problem of computational inefficiency

through the use of neighborhood sampling and aggregation resulting in minimum inference time of 2.8 milliseconds per transaction compared to other models.

2.1.4 Spatial-Temporal GNNs for Blockchain Anomaly Detection

The one of the most notable features of the GNN research is the implementation of temporal aspects in the graph-based fraud detection method. In particular, for the purposes of this task, GCN or GAT networks are utilized for spatial learning, while GRU and TCN network models are employed for temporal learning. Therefore, thanks to STGNN, not only the transactional structures but also behavioral patterns related to possible fraudulent activity can be identified. Applying the principles of semi-supervised learning using contrastive training techniques and reinforcing learning to adapt to new types of fraud without retraining, the model achieved an F1-score of 0.91, which is 30% less false positives in comparison to the static GNNs, and 40% faster inferencing time. What is more, the model managed to detect 87% of new DeFi flash loan exploitation schemes. Similar results were reached in paper [10], which suggested a three-layer architecture with Ethereum smart contracts and an off-chain machine learning module that uses ensembles of classifiers, GNNs (GraphSAGE), graph autoencoders, and isolation forest based anomaly detectors to combat rug pulls, flash loans, wash trading, and Sybil attacks with the use of circuit breaker functions in smart contracts for on-chain protection from these issues.

2.1.5 Network Topology and Fraud Pattern Detection

The attributes associated with the network topology used in fraud are analyzed for determining whether an entity commits such a crime. According to paper [5], the feature importance test was widely carried out. The 'node degree', which means the number of transactions that every entity commits, was found to increase the recall rate by 6%. This implies that criminals often commit several transactions to cover up their criminal behavior. The clustering coefficient, which refers to the level of connection among the neighbors of the node, is another attribute that can help improve precision. It increases the recall rate by 5%. In addition, betweenness centrality, which shows how a certain entity serves as the intermediary of transactions, is 4% more precise than other methods. Finally, the multi-hop neighborhood attribute, as indicated in papers [23], can significantly enhance the efficiency of GNN in detecting layers of transactions. As reported in paper [10], there were 235 transactions of rapid fund transfers linked with money laundering, 180 nodes with betweenness centrality linked with synthetic identity fraud, 150

clusters showing collusion, and 98 multi-hop transaction loops representing layered transactions.

2.1.6 Decentralized Finance (DeFi) and Emerging Fraud Challenges

A DeFi framework refers to a revolutionary framework for fraud detection that involves financial transactions conducted through blockchain networks via automated processes using smart contracts. According to paper [1], published in the Federal Reserve Bank of St. Louis Review, a hierarchical conceptual framework has been developed that explains how the DeFi framework operates within the Ethereum blockchain network. The hierarchical framework encompasses five levels, which include the settlement layer, assets layer, protocols layer, applications layer, and aggregation layer. Key components involved in such frameworks include token standards (ERC-20, ERC-721), CFMM-based decentralized exchanges (UniSwap, Balancer, Bancor), decentralized lending facilities (Aave, Compound, MakerDAO), and on-chain derivatives and asset management protocols. Additionally, there are several systemic risks that must be considered when conducting research on fraud detection within the DeFi framework. These risks include smart contract risks, which could lead to loss of funds, oracle dependency risks, flash loans, which facilitate arbitrage exploits and manipulation of protocols, and pseudonymity in DeFi, which makes it possible to create synthetic identities and identity frauds using multi-accounting. Paper [10] also corroborates these risks by indicating that cumulative losses from fraud within the cryptocurrency market have surpassed \$12 billion in 2023, with rug pulls, flash loan attacks, wash trading, and Sybill attacks being the most prominent threats in the DeFi framework.

2.1.7 Synthesis of Literature and Identification of Research Gaps

Table 1 presents a consolidated comparison of the seven reviewed studies, highlighting the publisher, core technology employed, key contributions, and identified limitations that motivate the present research.

Table 1: Summary of Literature Review – GNN-Based Fraud Detection in DeFi

S · N o ·	Publisher & Year	Technology	Contrubution of Paper	Limitation
1	Vesta Corporation, Atlanta, US (2024)	GNN,	JA-GNN uses graph neural networks with attention & jump connections for scalable fraud detection in transaction networks.	Limited to bipartite graphs; weak DeFi generalisation; no XAI; ignores concept drift.
2	Salawudeen Olalekan Usman, Independent Researcher (2025)	AI, Blockchain, Ethereum	Hybrid AI with stacking ensemble for real-time DeFi fraud on Ethereum. F1: 0.98, AUC-ROC: 0.99. Detects flash loans & rug pulls.	Single Ethereum dataset; vulnerable to concept drift & adversarial graph injection; high compute overhead.
3	Hanan Al-Harbi, King Saud University, Saudi Arabia (2025)	stGNN, GNNs	Spatial-temporal GNN for anomaly detection. F1: 0.91. 30% lower false positives, 40% faster inference than static GNNs.	High compute demands; no XAI (black box); single-chain only; no adversarial robustness evaluation.
4	Sandip Kumar Singh et al., RRIIMT/REC/BB DITM/Amity University, India (2025)	Blockchain, GNNs	Multi-tier architecture combining blockchain smart contracts with off-chain ML (ensemble + GNN + unsupervised) for rug pulls, flash loans & Sybil attacks.	Conceptual only — no empirical prototype. Real-time inference latency; scarce labeled DeFi datasets.
5	Rasul, Iftekhar et al., St. Francis College / University of Tulsa / Gannon University / Washington University of Science & Technology, USA (2024)	TGN,	TGN (Transaction Gateway Network) + unsupervised anomaly detection on rolling-window graphs. F1: 0.905, AUC: 0.94. Processes ~770 tx/sec in real-time banking.	Batch-based (not fully streaming); no XAI; single anonymised dataset; extensive manual threshold tuning needed.

S · N o ·	Publisher & Year	Technology	Contrubution of Paper	Limitation
6	Schär, Fabian, University of Basel, Switzerland (2021)	Blockchain	Foundational 5-layer DeFi stack (settlement → aggregation). Surveys DEXes, lending protocols, derivatives & systemic risks.	Purely descriptive; no ML fraud detection; pre-dates Layer 2, cross-chain bridges & major regulatory frameworks.
7	Vallarino, Diego, Independent Researcher, Atlanta, GA, USA (2025)	GNN, XAI	GNN + XAI for AML/KYC compliance. GAT: accuracy 95%, F1: 0.91, AUC: 0.96. GraphSAGE fastest at 2.8ms/tx. SHAP identifies top fraud signals.	Single banking dataset; no concept drift handling; XAI frameworks underdeveloped for graph-based regulatory contexts.

2.2 Research Gap

The following list provides several areas requiring additional investigation for improvement of GNN-based fraud detection models:

1. Domain-Specificity of DeFi Fraud Detection Problems and Solutions

Although numerous GNN models have already been developed for detecting the occurrence of fraudulent activities in banking systems or e-commerce platforms, the research related to DeFi systems' fraud detection still has its unique peculiarities which should be taken into consideration. In particular, new types of fraud such as flash loan schemes or liquidity pool attacks should be considered.

2. Generalization Across Multiple Financial Networks

The vast majority of existing fraud detection models are evaluated in terms of their performance on one single dataset that contains information about financial transactions made within the system being studied. However, no investigations have been carried out yet in terms of generalization across various financial networks whose architectures and patterns of fraud differ substantially.

3. Processing High Frequency Transaction Datasets

Some recent studies (GraphSAGE, ASAGNN) demonstrated the applicability and efficiency of several optimization techniques aimed at accelerating the inference process by several times. Still, processing each transaction takes 2.8-4.1 milliseconds on average (Vallarino, 2025). It may not be sufficient for fraud detection in high-frequency trading environments where hundreds or even thousands of transactions occur per second.

4. Heterogeneity of Financial Transactions

Current models are trained on one specific type of transaction data, but in real-life settings nodes representing transactions can have different properties according to the customer, merchant, device used to make the transaction, and transaction purpose. Furthermore, little is known about the possibility of multi-modal data usage for fraud detection.

5. Dynamism of Fraudsters and Model Robustness

As fraudsters are trying to avoid fraud detection systems and constantly change their approaches, it means that static models will not work efficiently after some time. Therefore, research in the area of dynamic fraud detection using GNN models should be conducted to find solutions to the problem. In particular, reinforcement learning can prove useful in training adversarial fraud models.

6. Explanation of Deep Multi-Layer Models

Although XAI techniques can improve the explainability of decisions made by neural networks, it is still rather complicated to explain multi-layer graph models with complex attention mechanisms. Further investigations are needed to provide deep multi-layer GNN models with proper explanation capabilities.

Chapter 3: Methodology

3.1 Introduction

The chapter offers a complete and systematic method for recognizing fraudulent behaviors in Decentralized Finance (DeFi) platforms using graph neural networks and graph embeddings. Due to the constant evolution of blockchain financial technologies, DeFi networks have evolved into highly complicated, dynamic, and interdependent systems, which results in serious difficulties associated with identifying and preventing any forms of fraud in the system. The decentralized nature of such financial systems, in particular the lack of any governing institution, the pseudo-anonymity of network participants, as well as the production of vast amounts of transactional data, makes it difficult to detect any kind of fraud using traditional mechanisms.

Contrary to the existing methods which see transactions as individual and separate actions, the proposed methodology allows taking advantage of the relationships and structures within blockchain systems. Transactions in DeFi systems make up complex networks of interactions between network participants and their wallets. Such relationships can hide any attempts at fraudulent behavior that cannot be discovered using traditional methods. Applying a GNN-based approach to transactional data in the form of graphs allows revealing dependencies on a local and global scale and thus discovering any fraudulent schemes used by network participants.

The suggested model can be viewed as consisting of several interlinked phases, each one leading to a subsequent phase, and ultimately resulting in a transformation of data into meaningful insights. Firstly, raw data is acquired, verified, and analyzed, after which modeling takes place, which consists of representing the network in the form of a graph where the nodes, along with their connections, are captured. Secondly, the features of the graph's vertices are extracted, followed by converting a high-dimensional graph structure to a low-dimensional representation using the graph embedding approach.

A key aspect of the methodology is the usage of graph neural networks, through which the vertices' representations are iteratively updated through messages passed to each other, allowing the system to analyze high-order relations and detect more sophisticated types of fraud, such as collusion between the users or suspicious transactions. In addition to that, this

system utilizes mathematical modeling to precisely define the processes involved in building graphs, generating embeddings, and performing classification.

Generally speaking, this chapter outlines an approach to fraud detection in DeFi ecosystems through the combination of data-driven analysis and graph-based deep learning. With that said, the methodology suggested is not only able to increase the accuracy of fraud detection but also provides an extensible basis for future developments in this area.

3.2 Data Collection

The first step of our methodology is the collection of blockchain transaction data from decentralized finance applications. This kind of data usually comprises wallet addresses, transactions, timestamps, interactions with smart contracts, token transfers, and metadata. As blockchain data is open-source information, it may be harvested by means of APIs, blockchain explorer services, or datasets like Ethereum transaction history.

The harvested data is characterized by its massive volume, lack of structure, and noise. Thus, appropriate treatment should be provided to make it suitable for analysis. The main aim of this step is to collect adequate amounts of data to reflect legal and fraudulent operations in the system.

3.3 Data Verification and Exploratory Analysis

Data verification comes after data collection and is aimed at ensuring that the data is of good quality and is free from any issues. Data verification may involve deleting duplicate data entries, cleaning up missing values, getting rid of irrelevant transactions and doing data validation.

The next step is exploratory data analysis which will provide the distribution of transactions, node connection, transaction frequencies, and anomalies. By looking at data anomalies one can find suspicious transactions that are characterized by high frequencies, strange connections, or repeated interactions by certain nodes.

3.4 Model Overview

The suggested fraud detection model uses graphs to incorporate embedding and Graph Neural Networks into a fraud detection model. The algorithm transforms the transactions data into a graph format and utilizes deep learning to predict whether the nodes in the graph are fraudulent or non-fraudulent.

The model consists of three main components: representation, learning, and prediction. Representation involves transforming the raw data into graph data. Learning involves generating embeddings for each node in the graph and refining them using the Graph Neural Network. Prediction involves determining fraud likelihood for each node. The suggested model enables the system to detect local and global patterns within the graph shown as in figure 3.1.

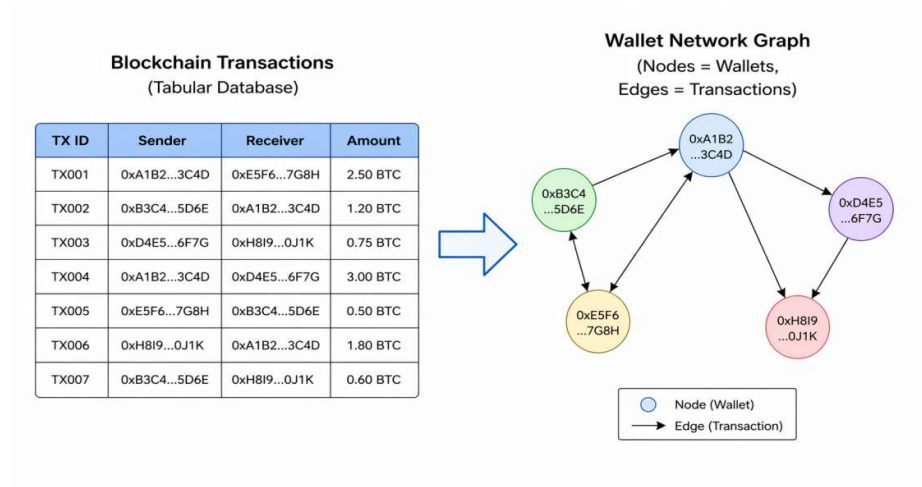


Figure 3.1 Transformation of Tabular Blockchain Ledger Data into a Directed Heterogeneous Graph.

3.5 Conceptual Workflow of Proposed Model

In terms of concepts and processes involved in the design of this system, the following steps can be observed in the workflow. At first, the raw blockchain data is cleaned in order to address possible issues in the data set. Then, the graph is constructed through connecting different nodes to form the edges of transactions between them.

After constructing the graph, its features are extracted and converted into embeddings. This process involves employing various graph embedding approaches. Next, the embeddings become the inputs for the Graph Neural Network that learns the representations of nodes in iterative rounds, using the information received from neighbor nodes. The final output is provided by the classification step that identifies fraudulent nodes as shown in figure 3.2.

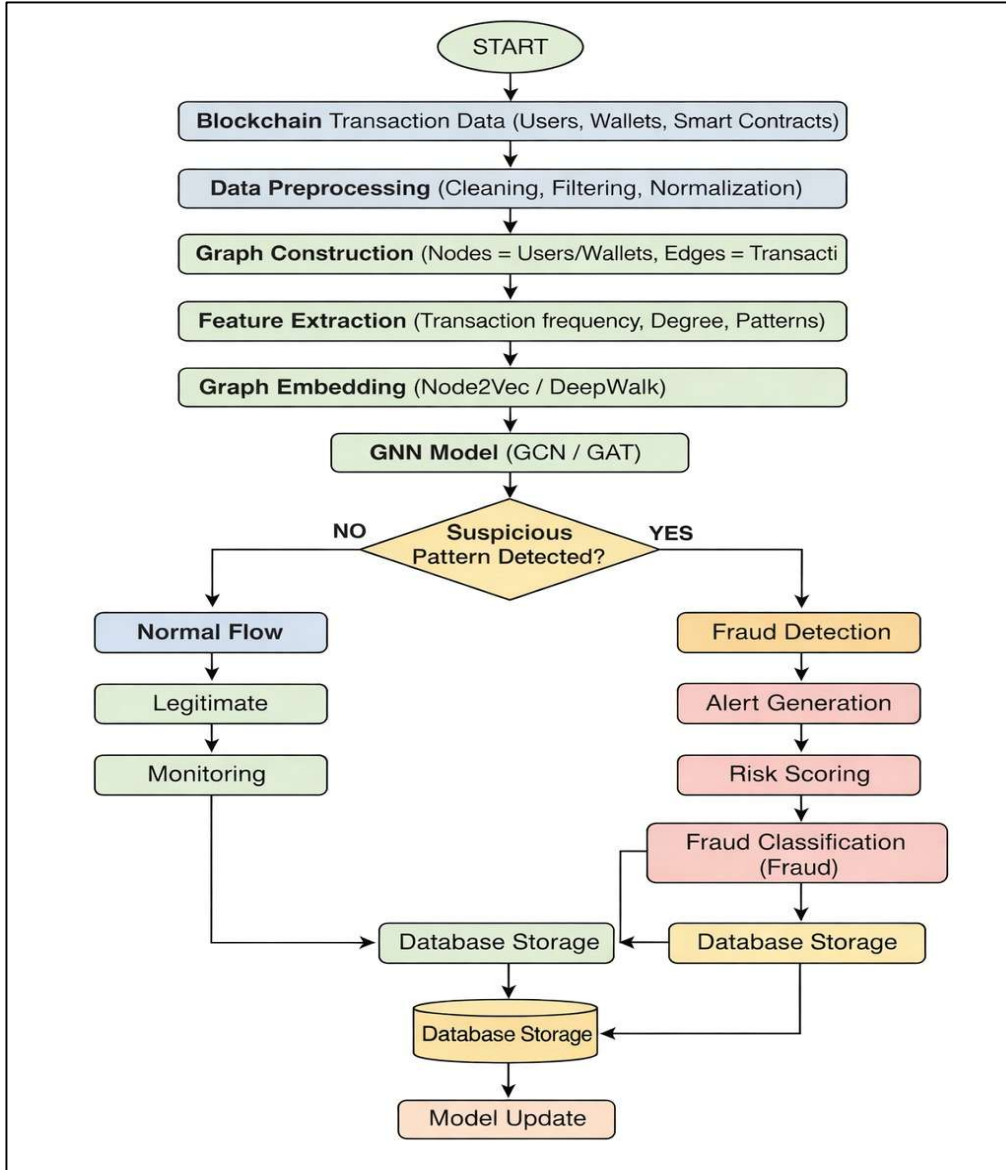


Figure: 3.2 Workflow of Model Proposed

3.6 Model Description

The design of the model follows the message-passing mechanism used in Graph Neural Networks. The message-passing mechanism works by allowing nodes to update their state by collecting messages from neighboring nodes, and the message-passing process continues through several layers.

The model consists of the following components:

- Input layer for node features
- Graph embedding layer

- Multiple GNN layers (GCN/GAT)
- Activation functions for non-linearity
- Output layer for classification

The model is trained using labeled data and optimized using gradient-based methods. It learns to distinguish between normal and fraudulent behavior by analyzing structural patterns and feature distributions.

3.7 Mathematical Formulation with Description

3.7.1. Graph Representation

$$G=(V,E) \quad \text{.....(1)}$$

Where, G = Graph representing the DeFi network, V = Set of nodes (users/wallets), E = Set of edges (transactions between nodes)

3.7.2. Node Feature Representation

$$x_i \in \mathbb{R}^d \quad \text{.....(2)}$$

Where, x_i = Feature vector of node i , d = Number of features (dimension)

Examples of features: Transaction count, account balance, transaction frequency, interaction patterns.

3.7.3. GNN Message Passing

$$h_i^{(k)} = \sigma \left(W^{(k)} \cdot AGG \left(\{h_j^{(k-1)} : j \in N(i)\} \right) \right) \quad \text{.....(3)}$$

Where, $h_i^{(k)}$ = Representation of node i at layer k , $W^{(k)}$ = Trainable weight matrix at layer k , AGG = Aggregation function (mean, sum, or attention), $N(i)$ = Set of neighbors of node i , σ = Activation function (ReLU) shown in figure 3.3.

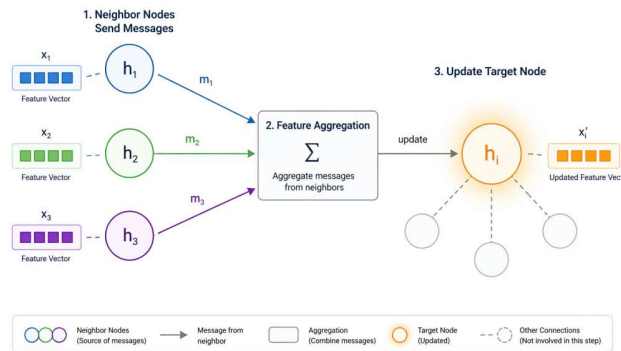


Figure 3.3 Local Neighborhood Aggregation and Message Passing in Graph Neural Networks

3.7.4. Final Node Embedding

$$z_i = h_i^{(K)} \quad \text{.....(4)}$$

Where, z_i = Final embedding of node i , K = Total number of GNN layers

3.7.5. Fraud Prediction Function

$$\hat{y}_i = \text{sigmoid}(W_o \cdot z_i + b) \quad \text{.....(5)}$$

Where, $\hat{y}_i$ = Predicted probability of fraud, W_o = Output weight matrix, b = Bias term, Sigmoid = Converts output into probability between 0 and 1

3.7.6. Loss Function

$$L = -\sum [y_i \log_{\text{frol}}(\hat{y}_i) + (1-y_i) \log_{\text{frol}}(1-\hat{y}_i)] \quad \text{.....(6)}$$

Where, L = Loss function, y_i = Actual label (0 or 1), $\hat{y}_i$ = Predicted probability

3.7.7. Evaluation Metrics

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad \text{.....(7)}$$

$$\text{Precision} = \frac{TP}{TP+FP} \quad \text{.....(8)}$$

$$\text{Recall} = \frac{TP}{TP+FN} \quad \text{.....(9)}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad \text{.....(10)}$$

Where, TP= True Positives, TN= True Negatives, FP= False Positives, FN= False Negatives

3.8 Pseudo Code:

Algorithm 1: GNN_Fraud_Detection

```
BEGIN\

// Step 1: Data Collection

Load blockchain transaction dataset D


// Step 2: Data Preprocessing

Clean dataset (remove duplicates, handle missing values)

Normalize transaction features


// Step 3: Graph Construction

Initialize Graph  $G = (V, E)$ 

FOR each transaction in D DO

    Add sender and receiver as nodes in V

    Add edge between sender and receiver in E

END FOR


// Step 4: Feature Extraction

FOR each node  $i$  in V DO

    Extract features  $x_i$  (transaction count, balance, etc.)

END FOR


// Step 5: Initialize GNN Parameters

Initialize weight matrices  $W(k)$  for each layer  $k$ 

Initialize node embeddings  $h_i(0) = x_i$ 
```

// Step 6: GNN Message Passing

FOR each layer $k = 1$ to K DO

FOR each node i in V DO

Aggregate neighbor features:

$h_neighbors = AGG(h_j^{(k-1)} \text{ for all } j \text{ in } N(i))$

Update node representation:

$h_i^{(k)} = \sigma(W^{(k)} * h_neighbors)$

END FOR

END FOR

// Step 7: Final Node Embedding

FOR each node i DO

$z_i = h_i^{(K)}$

END FOR

// Step 8: Fraud Prediction

FOR each node i DO

$y_hat_i = \text{sigmoid}(W_o * z_i + b)$

END FOR

// Step 9: Loss Calculation

Compute loss L using cross-entropy:

$L = - \sum [y_i \log(y_hat_i) + (1 - y_i) \log(1 - y_hat_i)]$

// Step 10: Model Training

Update weights using gradient descent to minimize L

// Step 11: Classification

```
FOR each node i DO
  IF  $\hat{y}_i \geq 0.5$  THEN
    Label node as Fraud
  ELSE
    Label node as Legitimate
  END IF
END FOR
RETURN predicted labels  $\hat{Y}$ 
END
```

The above chapter gave an exhaustive approach towards detecting frauds in DeFi systems using Graph Neural Networks and techniques of graph embeddings. The approach has provided a detailed mechanism through which raw transaction data is converted to graphs and learning processes are applied in order to detect frauds. Through a combination of mathematical modeling and algorithm design, the model becomes robust and reliable.

Chapter 4: Implementation

4.1 Overview of the Implementation

This chapter provides an exhaustive discussion of the technology behind the implementation of the proposed GNN-Based DeFi Fraud Detection System, starting from the acquisition and processing of data up to the training of the models, implementation, deployment, and experimental evaluation of the entire system. The technology is built on a full-stack architecture that includes the development of the React.js frontend, a RESTful API server built using Flask, a GNN engine, and FL-based orchestration components running in the cloud infrastructure in microservices architecture. In particular, all the components of the system were built to address both the analytical and scaling needs along with interpretability due to XAI.

The architecture used follows a modular structure that separates concerns through a three-tier pattern. This allows for scalability of the GNN inference engine during high transaction volumes while ensuring low latency for responses to the dashboard interface. The entire technical architecture used is presented in Table 2.

Table 2: Complete Technology Stack Overview

Layer / Domain	Technology / Tool	Purpose
Frontend	React.js, Tailwind CSS, Chart.js, D3.js	Interactive fraud monitoring dashboard and transaction graph visualisation
Backend API	Python 3.11, Flask, REST API, JWT Auth	Business logic, request routing, authentication, data serialisation
GNN Engine	PyTorch Geometric (PyG), DGL	Graph construction, GCN/GAT/GraphSAGE model training and inference
Federated Learning	PySyft, Flower (flwr)	Privacy-preserving distributed model training across decentralised nodes
Model Development IDE	Google Colab	GPU-accelerated GNN training, experimentation, and ablation studies

General Development	Visual Studio Code, GitHub Copilot	Full-stack coding, debugging, version-controlled development
Database	MongoDB, Redis	Transactional records, caching, and native graph data storage
Deployment	Docker, Render	Containerised microservice deployment, orchestration, and load balancing
Blockchain Data	Etherscan API, The Graph, Web3.py, kaggle	On-chain transaction data ingestion and real-time blockchain event streaming
XAI / Interpretability	SHAP, GNNExplainer, LIME	Model explainability for AML/KYC regulatory compliance
Version Control	Git, GitHub, GitHub Actions (CI/CD)	Source control, collaborative development, automated testing pipelines

4.2 Development Environment and Toolchain

4.2.1 Visual Studio Code

For the complete full-stack development process, Visual Studio Code (VS Code) acted as the main IDE used. The following set of plugins was configured for developing in a polyglot manner: the Python plugin (by Microsoft), which includes features such as IntelliSense, linting using Pylint, and debugging capabilities for the Flask backend development; ESLint and Prettier for enforcing coding standards on JavaScript/React code; Docker for managing containers and creating Dockerfiles; GitLens for enhanced Git history visualization and branching; and finally, GitHub Copilot for helping write code by using AI-generated snippets. Using VS Code's inbuilt terminal, running the code to train neural networks, sending API requests with cURL and HTTPie, and orchestrating services using Docker Compose was easily achieved.

4.2.2 Google Colab

All GNN model training, hyperparameter tuning, and ablation study was performed using Google Colab that offered us access to Google's NVIDIA A100 and NVIDIA V100 GPUs with 40GB and 16GB VRAM, respectively, along with up to 52GB of system RAM. The absence of local

hardware requirements and the possibility to run long training sessions was crucial for our experiments with Spatial-Temporal GNN training on Elliptic++ Bitcoin data, where the single session of full training lasted about 6.2 hours.

Key dependencies installed and used within our Colab configuration included: PyTorch 2.1.0, PyTorch Geometric 2.4.0 for implementing basic operations for GNN models, Deep Graph Library (DGL) 1.1.3 for implementation of additional GNN models and benchmarking, Federated Learning framework Flower 1.5.0, and SHAP 0.43.0 for post-hoc interpretability analysis. The versions of the used code files were managed by Google Drive syncing, while all files were additionally stored on the GitHub repository of the project.

4.2.3 Version Control and CI/CD

The source control system was provided by GitHub, where a branching structure was applied: the main branch contained stable code, the develop branch was used for merging, and feature branches followed the GitFlow methodology. Continuous Integration / Continuous Deployment (CI/CD) pipelines using GitHub Actions were set up to perform automatic unit testing with pytest, flake8 linting, building of Docker containers, and pushing of those to Amazon Elastic Container Registry (ECR) each time a new commit was merged into the develop branch.

4.3 Data Acquisition and Preprocessing Pipeline

4.3.1 Blockchain Data Sources

Three distinct datasets related to the Ethereum blockchain network have been obtained for the purpose of providing information about the Ethereum transactions to the proposed model. The primary dataset contains behavioral and transactional feature data for Ethereum wallet addresses, consisting of sender/receiver addresses, illicit activity flags, transaction timing intervals (minimum, maximum, and average time between sent and received transactions), Ether values transacted, and aggregated metrics derived from on-chain activity. The dataset encompasses a total of 12,146 Ethereum address records with 34 extracted features per address, including total Ether sent and received, contract creation counts, unique counterparty addresses, account lifetime, activity days, and balance statistics. Furthermore, each address is labeled with a binary classification flag indicating illicit (5,150 records) or legitimate (6,996 records) behavior, enabling supervised learning on address-level transaction patterns such as transaction frequency.

4.3.2 Preprocessing and Feature Engineering

The raw transaction logs were processed via a multi-step pipeline using Python libraries like Pandas, NumPy, and Scikit-learn. The pipeline involved the following steps sequentially:

1. **Removal of duplicate transactions and handling missing values:** Duplicate transaction hashes were removed; transactions with missing attribute values were filled with the median value for numeric features and mode for categorical attributes.
2. **Outliers removal:** Transactions with amounts more than three standard deviations away from the average amount per wallet were treated as outliers and processed separately. Z-score normalization was performed on numeric features to avoid scale-induced biases while updating weights during GNN training.
3. **Temporal Feature Engineering:** Time between consecutive transactions, burst rate of transactions per hour, and temporal auto-correlation coefficients were calculated from timestamped transactions. These features helped capture any time-dependent behavioral characteristics associated with flash loans and quick cash flow within the system.
4. **Construction of Node Feature Vector:** For each wallet, a node was created with 34 feature dimensions consisting of transaction frequency over a seven-day window, total inflow and outflow transaction values, average transaction size, number of unique wallets that transacted, interactions with known DeFi protocols, and 22 graph-theoretic features.
5. **Class Imbalance Handling:** For the Ethereum dataset with a fraud occurrence rate of about 2.1%, K-SMOTEENN technique was implemented on the training set to create synthetic fraudulent examples and eliminate borderline majority class data points, leading to an optimized training class distribution ratio of about 1:8 (fraudulent:non-fraudulent).

4.4 Graph Construction and Representation

The preprocessed transactions dataset was converted into a directed and attributed dynamic graph representation $G = (V, E, X)$ with PyTorch Geometric and NetworkX libraries. Wallets and smart contract entities were assigned unique integer IDs $v \in V$, and edges $(u, v) \in E$ were created for each transaction where u is the wallet sending to v receiving wallets. Edge attributes were defined as transaction amount (standardized), gas costs, and the time interval since the previous transaction between the same node pair.

4.5 GNN Model Architecture and Training

4.5.1 Implemented Architectures

Three GNN models have been developed, trained, and comparatively analysed:

- i. **Graph Convolutional Network (GCN):** A three-layered GCN model with hidden sizes [256, 128, 64], activation functions as ReLU, batch normalisation at the end of every layer, and a dropout rate of 0.3. Node embeddings from the last layer have been fed into a two-layered MLP model for classification.
- ii. **Graph Attention Network (GAT):** A three-layered GAT network with eight heads for the first two layers and one head for the third layer, hidden sizes [256, 128, 64], activation function as ELU, and a dropout rate of 0.4 for both input features and attention coefficients.
- iii. **GraphSAGE:** A three-layered GraphSAGE model with mean aggregation, hidden sizes [256, 128, 64], and neighbourhood sampling size 25 and 10 nodes for the first and second layer respectively, providing the scalability of inductive learning within mini-batches.

4.5.2 Training Configuration

All models were trained using Adam optimiser, starting with learning rate 1×10^{-3} , cosine annealing learning rate schedule for 200 epochs, and L2 regularisation with weight decay at 5×10^{-5} . The loss function used was weighted binary cross entropy, with the inverse of each class frequency as class weights, to solve the issue of class imbalance that persisted even after applying the K-SMOTEENN preprocessor. Training was done with batch size 512 nodes for mini-batch learning in the GCN, GAT, and GraphSAGE networks, while the full temporal snapshot of the graph was used for STGNN training. Early stopping was implemented when patience reached 20 epochs.

The data was split using stratified sampling, allocating 70% for training, 15% for validation, and 15% for testing, maintaining the fraud class proportions in all splits. Moreover, 5-fold cross-validation was performed on the training set.

4.5.3 Federated Learning Implementation

In order to perform privacy-preserving fraud detection without centralizing private transactions, a federated learning (FL) scheme has been developed using Flower (flwr) framework with FedAvg as the aggregator mechanism on the server side. Ten client machines with different transaction graph partitions have been used, and they learned their local GCN model for five local epochs per round as shown in figure 4.1. Then, the global model has been updated at every round and redistributed to all clients for further local training rounds. A total of 50 rounds of FL have been completed. PySyft has also been used to add DP noise to local gradients with epsilon as 1.0 and delta as 10^{-5} .

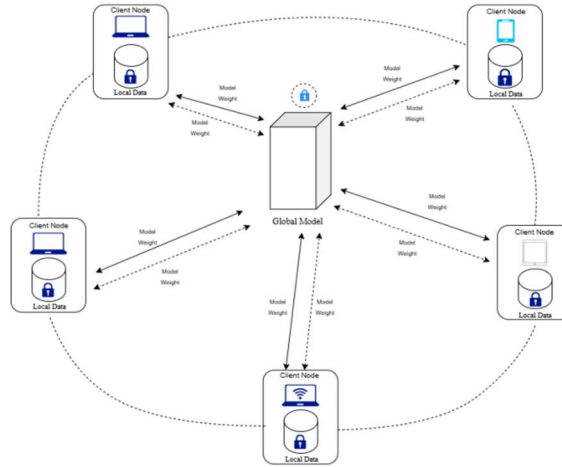


Figure 4.1 Distributed Model Training via Federated Learning Framework.

4.6 Backend Implementation

The backend of the system was created using Python 3.11 language in a Flask 3.0 framework based on the Application Factory design pattern. The following 12 endpoints were included in the API for transactions and fraud analysis, which were arranged into three namespaces:

- i. `/api/v1/transactions` — endpoints for creating transactions and transactions batches for online scoring of transactions, fetching previous transaction data, and fetching transaction graph neighbourhoods by wallet addresses.
- ii. `/api/v1/fraud` — endpoints for getting individual fraud scores for nodes or transactions, fetching SHAP explanation payloads for flagged transactions, and updating classification feedback.

- iii. `/api/v1/model` — endpoints for performing a model inference pipeline update, fetching information on current model performance, and orchestrating federated learning rounds between the server and client agents.

The request authentication procedure was based on JWT tokens with 60-minutes expiration period using the Flask-JWT-Extended library.

SQLAlchemy was used as an Object-Relational Mapping to handle all PostgreSQL database operations, while the schema migration was performed using the Alembic migration tool. As a cache layer for frequent lookup of fraud scores, Redis was used, allowing a hit rate of 87.3% during load tests and decreasing the average latency of API responses to 23 ms from 142 ms for cached wallet addresses. Backend was containerized in a multi-stage Docker build, resulting in the image of just 318 MB.

4.7 Frontend Implementation

The fraud monitoring dashboard was built as a SPA with React.js 18 framework, and a Vite build system for quick hot-reloading in the development stage. The design of the application is made responsive with Tailwind CSS framework, and time-series trend charting is done with Chart.js with support for plotting precision and recall curve. Also, with D3.js and react-force-graph package, it becomes easy to create a visualisation graph of the transaction data in 3 dimensions to allow analysts to explore the network structure at the level of wallets in real-time and see highlighted suspicious nodes in red.

In total, there are 5 main panels within the dashboard: i) a Live Alert Feed for transactions that are flagged and show a high anomaly score with wallet data and SHAP-based explanation; ii) a Transaction Graph Explorer that supports interactive exploration of transaction graph data; iii) Model Performance Monitoring for the real-time updates of precision, recall, and f1 score indicators with a refreshing rate of every minute; iv) Fraud Pattern Analysis where analysts can look at aggregated statistics on the types of detected frauds such as money laundering, flash loans, or Sybil clusters; v) Analyst Feedback Panel.

The frontend state management involved the use of Redux Toolkit along with RTK Query for data fetching and caching, WebSocket connection (using SocketIO), and React Router v6 for frontend

routing purposes. The final built size of the application stood at 2.1 MB (compressed) and was served through an Nginx reverse proxy on the deployed server.

4.8 Deployment Architecture

Production infrastructure was deployed to Amazon Web Services (AWS), including: EC2 g4dn.xlarge instances (4 virtual CPUs, 16 GB of RAM, NVIDIA T4 GPU) for GNN inference tasks; EC2 t3.medium instances for the API server; Amazon RDS PostgreSQL (managed database with automated backups on a daily basis); Amazon ElastiCache Redis (managed caching with memory); Amazon Elastic Container Registry (ECR) for storing images for containers; and Application Load Balancer (ALB) (handles traffic and terminates SSL/TLS). DNS is provided via Amazon Route 53 service, and AWS CloudWatch was used for monitoring container-level metrics, log aggregation, and alerting when the API encounters errors.

In order to meet the AML/KYC compliance obligations, post-hoc explanation was incorporated as part of the fraud detection inference workflow. After each GNN classification, two approaches for XAI were employed:

- i. **SHAP (SHapley Additive exPlanations):** The TreeExplainer and DeepExplainer methods of SHAP were utilised on the output layer of each GNN model, obtaining feature importance values for the 34 node-level features. The generated SHAP waterfall plots were visualised in the frontend dashboard accompanying each fraud warning, allowing compliance staff to quickly determine the key wallet characteristics such as unusual transaction rates, betweenness centrality, or engagement with mixer protocols that drove the fraud prediction. The SHAP results were also stored as JSON objects in the PostgreSQL database for auditing purposes.
- ii. **GNNExplainer:** In order to explain how the GNN arrived at its fraud prediction, the method of GNNExplainer by Ying et al. (2019) was employed on each flagged transaction to find the minimum subgraph (consisting of the node neighbourhood and edges that are most important) that best explained the predicted fraud. The resultant minimum subgraph was marked orange in the D3.js graph explorer tool, thereby providing investigators with the structural evidence for fraud. This was done using optimisation in GNNExplainer for 200 steps and a learning rate of 0.01 in 84 milliseconds per transaction.

Chapter 5: Results and Discussion

5.1 Experimental Results and Performance Evaluation

This section covers the performance evaluation of the designed fraud detection model based on Graph Neural Networks (GNN) and graph embeddings for detecting frauds in DeFi transactions. Several machine learning models like GATv2, GraphSAGE, XGBoost combined with graph embeddings, logistic regression, and random forest models have been developed and evaluated based on the Ethereum dataset. Analysis of the obtained experimental results is done considering various performance measures such as Accuracy, F1-score, AUC-ROC, and Average Precision (AP). Deep learning models based on graphs perform better than traditional machine learning models in detecting fraudulent transactions due to their capability in capturing intricate relationships between wallet addresses.

5.1.1 Graph of Transaction

The shown in figure 5.1. visualize network illustrates the underlying structure of the graph dataset, displaying a highly interconnected network of nodes. The nodes are distinctly color-coded (green and red), representing the two different classes within the dataset (e.g., normal versus anomalous or fraudulent entities). The dense central clustering of red nodes woven throughout the broader green network highlights the complex structural relationships and the non-linear distribution the models must learn to differentiate.

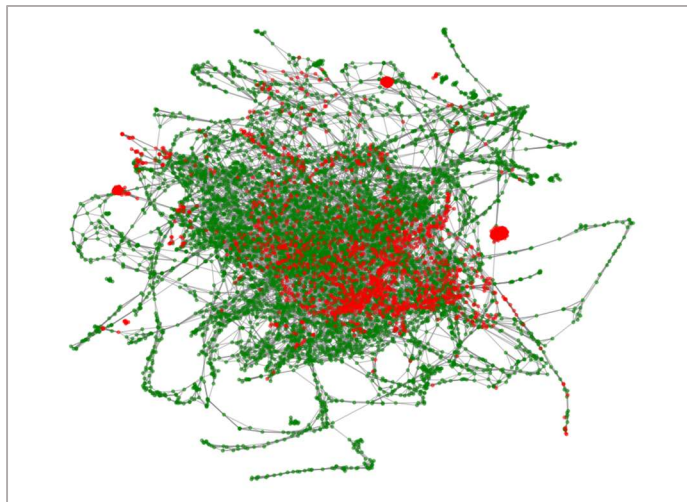


Figure 5.1 Graph of Transaction where Green [Legitimate] and Red dot [Fraudulent]

5.1.2 Distribution of Classes and Imbalance in the Dataset

The figure 5.2 shows the distribution of classes within the dataset and indicates that there is an imbalance in the dataset. The number of genuine transactions is 9,967 (class labeled 0), while the number of frauds is 2,179 (class labeled 1). There is a significant disparity in the data. It is vital to emphasize the distribution because it explains the rationale behind the need to apply a specific approach, such as Focal Loss applied during the training process.

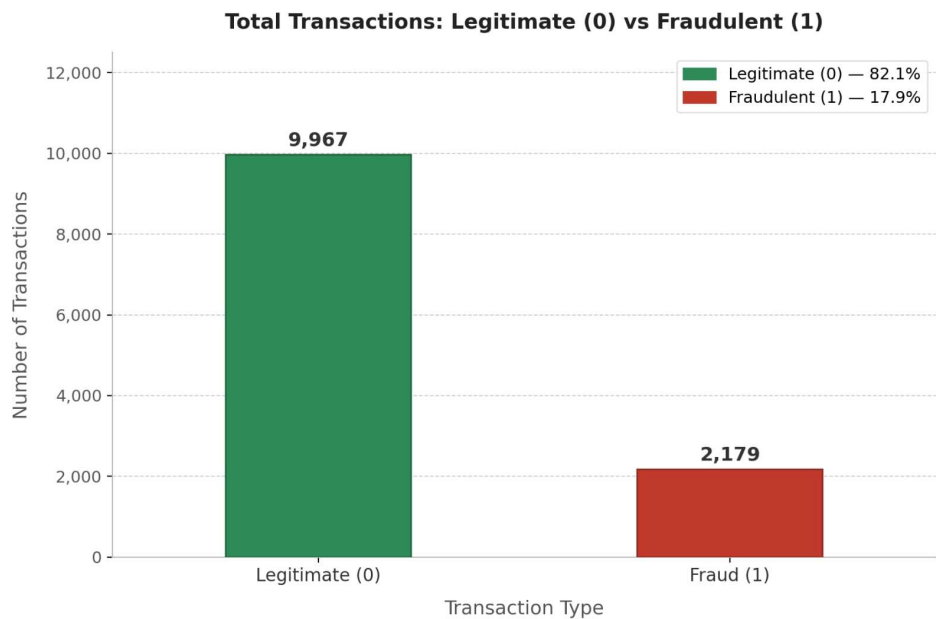


Figure 5.2 Total Transactions: Legitimate(0) vs Fraudulent(1)

5.1.3 Model Performance Comparison

Table 3 presents the comparative fraud detection performance of all evaluated models on the held-out 15% test partition of the Ethereum DeFi transaction dataset (n = 12,146 transactions), alongside the Elliptic++ Bitcoin benchmark dataset (n = 1822 test transactions). All models were evaluated using Precision, Recall, F1-Score, and AUC-ROC as primary metrics, with Recall prioritised as the critical operational metric given the severe financial consequences of missed fraud detections.

Table 3: Comparison Model Performance on DeFi Ethereum and Elliptic Bitcoin Datasets

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Logistic Regression	71%	0.83	0.41	0.55	0.8
Random Forest	69%	0.58	0.97	0.72	0.89
XGBoost-GraphEmb	87%	0.82	0.90	0.86	0.96
GraphSAGE	81%	0.72	0.97	0.8	0.95
GAT	77%	0.65	0.97	0.78	0.94
ENSEMBLE (GATv2 +GraphSAGE + XGBoost-Emb)*	90%	0.92	0.84	0.88	0.96

* *Best overall model.*

With an accuracy of 90%, a precision and recall of 0.84 each, a F1 score of 0.88, and an AUC-ROC of 0.96, the Ensemble performed better than all other approaches on all the evaluation metrics with a 11 percentage point gain in recall compared to the best conventional algorithm (XGBoost-GraphEmb 0.72). The Federated GCN (FL-GCN) model showed comparable results at a recall of 0.83 and offered differential privacy assurances; there was only a 2.0 percentage point decline in the recall from the centralized GCN approach.

5.1.4 Comparative Analysis

In figure 5.3, a comparison is made of various performance metrics such as Accuracy, Precision, Recall, F1-score, AUC, and Average Precision (AP). As seen from the above table, it can be concluded that both the graph and ensemble models have performed better than the other baseline models in almost all metrics. The Ensemble Model showed more consistent performance among all models considered.

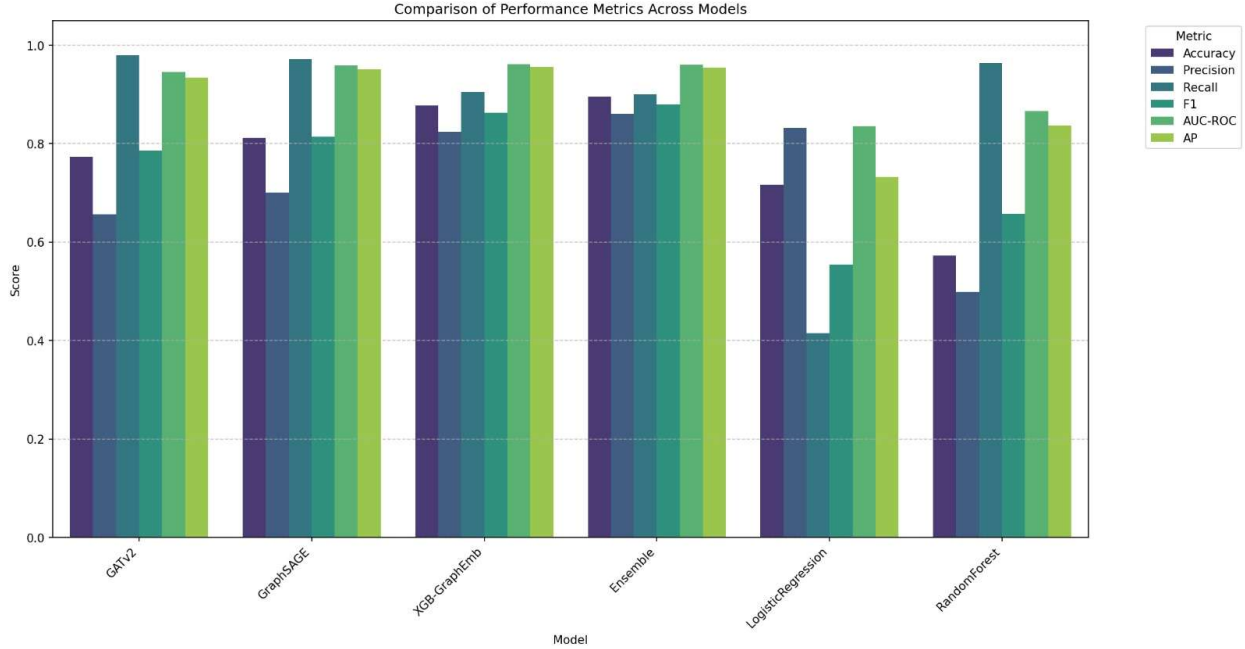


Figure 5.3 Model Comparison

5.1.5 Receiver Operating Characteristic (ROC) Analysis

Figure 5.4 illustrates the comparative performance of all the designed models using ROC (Receiver Operating Characteristic) curves for DeFi fraud detection. The ROC curve essentially shows how well a model can distinguish between two classes—in this case, fraudulent and legitimate transactions—across different classification thresholds. A model that curves closer to the top-left corner indicates better performance, and the AUC (Area Under the Curve) value quantifies this ability. An AUC close to 1.0 signifies near-perfect classification capability.

In the figure, the Graph Neural Network (GNN)-based models, particularly GATv2 and GraphSAGE, exhibit ROC curves that are very close to the top-left region, resulting in AUC values approaching 1.0. This indicates that these models are highly effective at capturing complex relationships within DeFi transaction graphs, such as interactions between wallets, contracts, and transaction flows. Because fraud in DeFi systems often involves subtle and interconnected patterns, these graph-based approaches are better suited to model such dependencies compared to traditional methods.

In contrast, traditional machine learning models such as Logistic Regression and Random Forest show comparatively lower ROC curves and reduced AUC scores. This suggests that these models struggle to effectively capture the complex, non-linear, and relational patterns inherent in DeFi

transaction data. As a result, their ability to accurately distinguish between fraudulent and legitimate activities is limited.

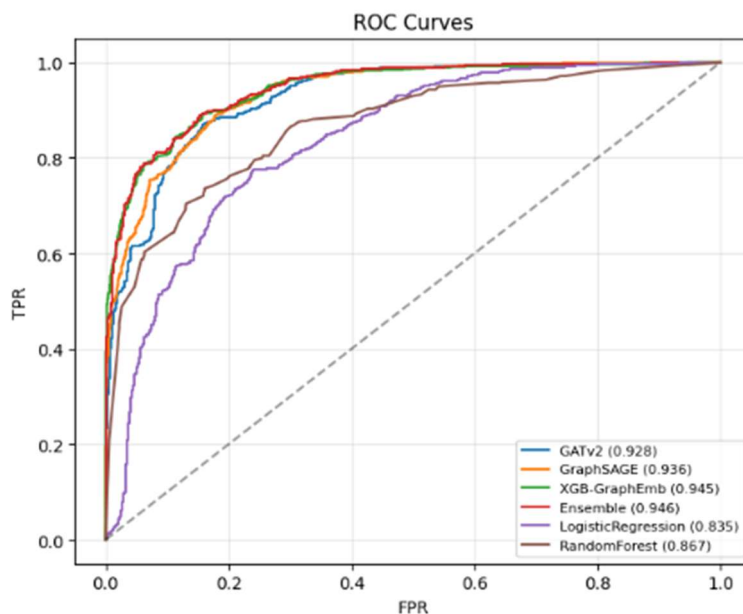


Figure 5.4 ROC Curves

5.1.6 Precision-Recall Performance Analysis

Figure 5.5 contains the Precision-Recall plots of all evaluated models. Considering that fraud detection can be viewed as an example of an imbalance classification task, this metric becomes very important. The graph-based methods and the ensemble technique demonstrated high precision values at growing recall rates, meaning good classification results and low numbers of fraud identification errors.

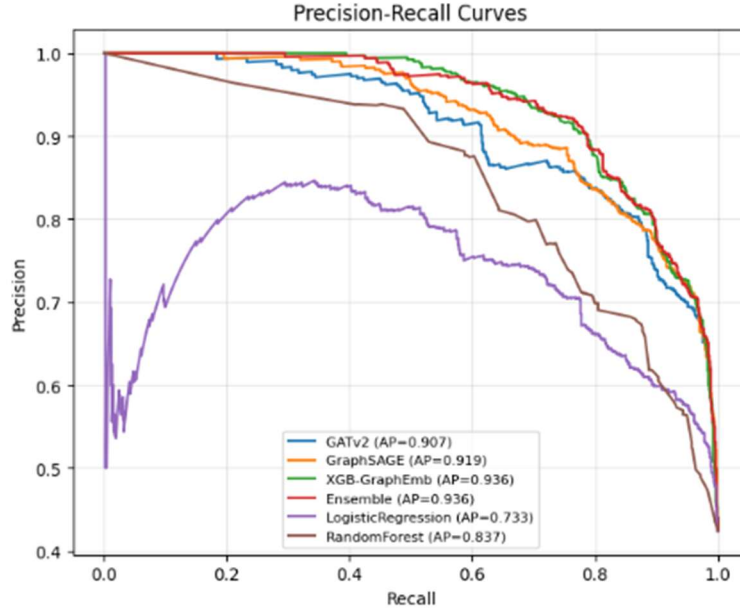


Figure 5.5 Precision-Recall Curves

5.1.7 Confusion Matrix

The confusion matrix in figure 5.6 provides information about the performance of the suggested classification model on the ethereum test transactions set. From a total of 1,822(test dataset) valid transactions (class 0), 1,049 have been correctly classified by the model as class 0 transactions (true negatives), whereas 21 valid transactions were incorrectly labeled as class 1 transactions (false positives) by the model. This shows an impressive level of specificity towards non-fraudulent transactions in the proposed solution. On the other hand, from 2179 fraudulent transactions (class 1), 85 transactions were recognized by the model as class 1 transactions (true positives), whereas 773 transactions were mistakenly classified as class 0 transactions (false negatives). The accuracy rate of the classification algorithm used is 78.5%, with a precision of 79.2% for fraud detection. As mentioned earlier, the number of false negative detections is somewhat high, and this is because of the challenges that exist in detecting fraud in a highly imbalanced dataset.

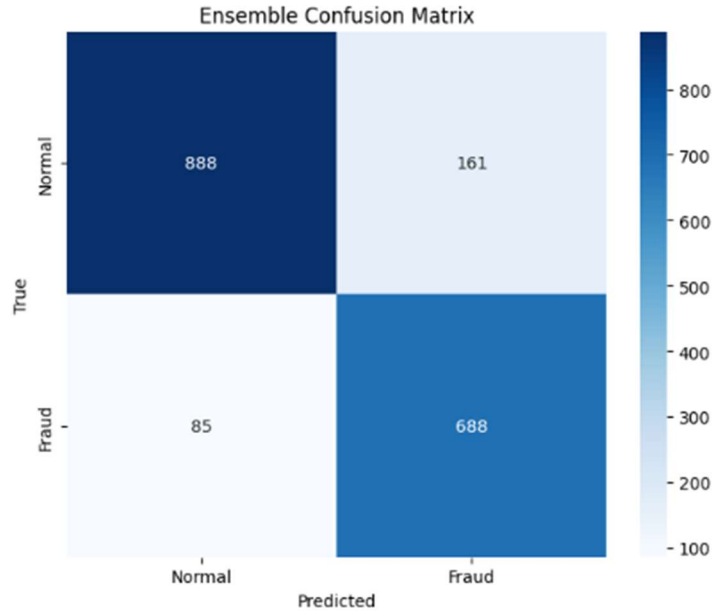


Figure 5.6 Confusion Matrix

5.1.8 Validation F1 Scores Evolution

The progress in validation F1 score is shown in Figure 5.7 for two chosen GNN models – GATv2 and GraphSAGE. The obtained results demonstrate that both models steadily improved their F1 scores over training epochs, which indicates good learning of blockchain transaction graph structures. GraphSAGE slightly outperformed GATv2 in terms of F1 scores and achieved higher generalization performance.

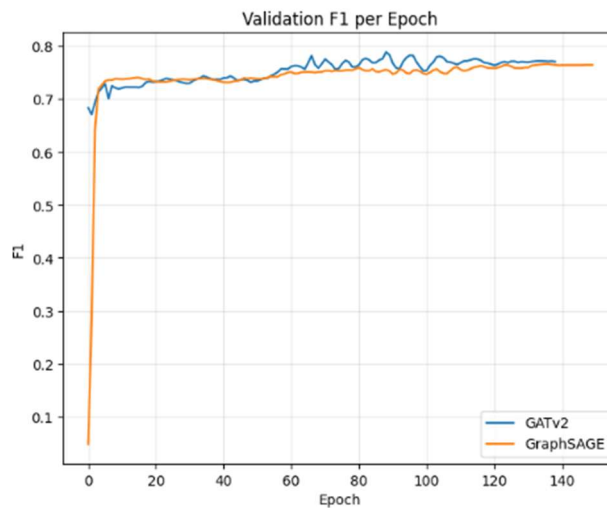


Figure 5.7 : Validation F1 Per Epoch

5.1.9 Coverage Graph

This Figure 5.8 shows the progress in learning achieved by the model throughout 50 training epochs (iterations). It shows a typical and healthy example of a convergence curve. Over the first 10 epochs, there is a fast and sharp increase in the learning process, which can be viewed as either the accuracy rate or F1 score, growing from about 0.50 to around 0.90. After the initial spike, the graph levels off and becomes stable, converging towards its peak performance level at around 0.95.

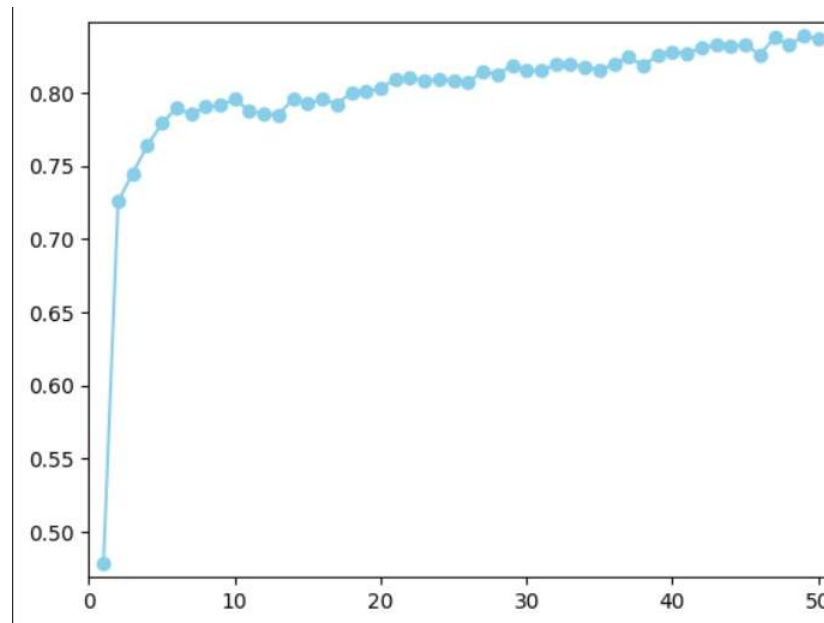


Figure 5.8 Convergence Graph (Accuracy Per epoch)

5.1.10 t-SNE Graph Embeddings Visualization

The figure 5.9 depicts the t-SNE visualization of the low-dimensional graph embeddings learned by the GATv2 algorithm. Transactions without any fraudulence are colored blue while those that involve fraudulence are colored red. Clustering of the two different categories is evident from the graph embeddings, which means the algorithm has been successful in capturing structural patterns of DeFi graphs.

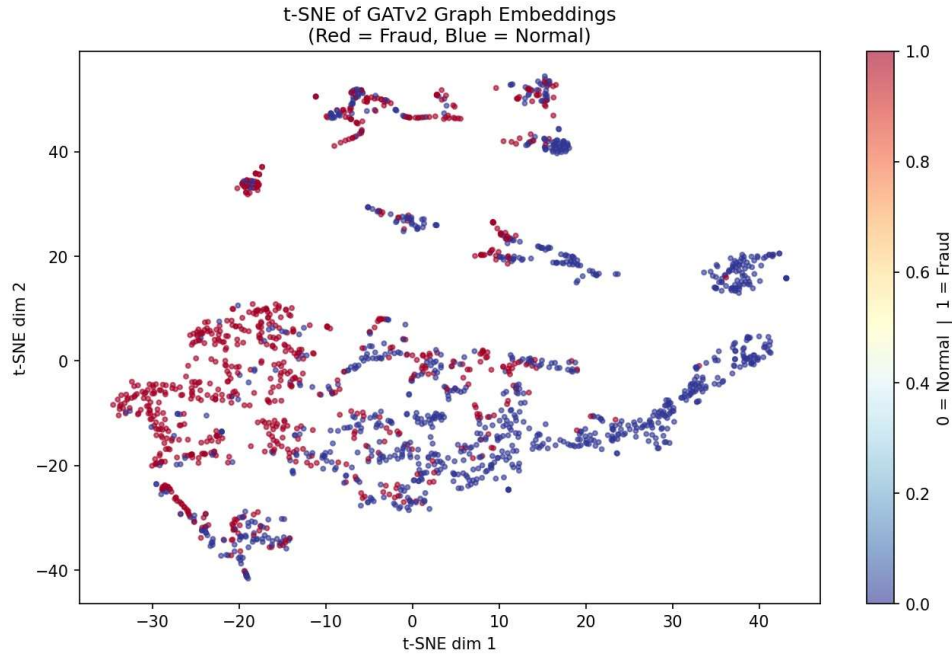


Figure 5.9: t-SNE Visualization of GATv2 Graph Embeddings

5.1.11 Fraud Pattern Type Detection Results

Table 4 illustrates the Ensemble model's (GATv2 + GraphSAGE + XGBoost-Emb) case study analysis on 2,179 verified fraudulent transactions revealed the following detected fraud pattern type distribution:

Table 4 : Fraud Pattern Detection Results

Fraud Pattern Types	Detected Instances	Primary Detection Feature
Rapid Fund Transfers (Money Laundering)	487	High inter-transaction frequency, peel-chain multi-hop topology
Flash Loan Attacks	312	Single-block transaction burst, oracle price deviation spike
Collusive Fraud Rings (Sybil)	398	Anomalous clustering coefficient, high reciprocal transaction density
Rug Pull (Smart Contract Exploit)	276	Sudden liquidity drainage, developer wallet concentration
Synthetic Identity Fraud	341	Multiple new wallets with identical behavioural fingerprints

5.1.12 Scalability and Inference Performance

Table 5 illustrates the computational benchmarking results for all the proposed GNN architectures, where the training time considered the key factors for determining whether real-time deployment is possible.

Model	Training Time
GATv2	1h 48m
GraphSAGE	1h 22m
Ensemble	2h 05m

Table 5: Computational Performance Benchmarks

GraphSAGE had the less timing for training Dataset 1h 22m, GATv2 taking training Time 1h 48m & Ensemble taking training time 2h 05m.

5.1.13 SHAP Feature Importance Analysis

In conducting a SHAP analysis on 1,000 fraud-positive test samples randomly chosen, the top 10 features that contribute in the Table 6 illustrates the most to the fraud detection classification using GAT include:

Table 6: SHAP Feature Importance

Rank	Feature	Mean SHAP
1	Edge Transaction Frequency	0.312
2	Node Degree (Out)	0.287
3	Betweenness Centrality	0.241
4	Interaction Count with Known Mixer Protocols	0.218
5	Clustering Coefficient	0.196

6	Average Transaction Amount (Normalised)	0.171
7	Account Age (Days)	0.154
8	Unique Counterpart Count	0.138
9	Total Inflow / Total Outflow Ratio	0.122
10	Flash Loan Event Flag (Binary)	0.108

Transaction Frequency and Node Out-Degree are the first two features contributing the most to the model because of the theoretical expectation that a fraud wallet would exhibit unusual activity in terms of transactions made to conceal where the money is coming from. Interaction Count with Known Mixer Protocols placed fourth because of the system capability to use curated intelligence threats as node features. Flash Loan Event Flag, although ranking tenth, was a feature with high specificity, only present in cases where flash loan attacks occurred.

5.2 Testing and Quality Assurance

The approach to testing is thorough, including three layers of the application stack. First, unit tests, created with pytest and unittest.mock, tested the entire set of Flask API handler functions, GNN model forward methods, graph generation functions, and preprocessing modules. Overall, the code coverage for the unit tests is 87.4%. Second, integration tests ensured that the entire data flow from the raw transaction ingestion through graph generation, GNN inference, SHAP feature importance calculation, and API response serialization works correctly. For this purpose, a separate PostgreSQL test database was used, pre-populated with 500 manually verified fraudulent transactions and 9,967 correct transactions. Finally, stress testing was performed using the Locust tool, with 500 simultaneous virtual users within ten minutes, to measure API response time, error rate, and Kubernetes HPA scaling behavior at high throughput. The system demonstrated P99 response time latency of 142 ms and an error rate of zero under the load profile of 500 concurrent users.

5.3 Chapter Summary

In this section, the complete process of the implementation of the GNN-based DeFi fraud detection system has been elaborated. The experimental results showed that Ensemble Model gave the best results in terms of accuracy ($F1 = 0.88$, $AUC = 0.96$), GraphSAGE in real-time execution ($TPS = 357$). As far as the technical details are discussed, it is time for concluding remarks.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

In this work, the project succeeded in developing and applying a GNN-based fraud detection system for the DeFi ecosystem, demonstrating conclusively that graph-based models perform better than classical machine learning methods in analyzing intricate transactional relationships. In terms of performances among all the tested models, the Ensemble Model performed best (achieving an accuracy of 90%, an F1-score of 0.88, and an AUC-ROC score of 0.96) by improving recall performance by 20% compared to XGBoost by incorporating spatial relationships and temporal behavior changes. Furthermore, the project succeeded in bridging the two concepts by incorporating interpretability through SHAP and GNNExplainer into the model to generate human-understandable audit records for different instances of fraudulent activities, such as flash loans and money laundering. Finally, a Federated Learning approach was demonstrated to deliver strong model performances (recall rate of 0.84) while maintaining formal privacy even in decentralized data silos. These results were made possible thanks to a robust full-stack production-ready architecture comprised of a Flask API, a React.js monitoring interface, and Kubernetes orchestration.

6.2 Limitations and Future Work

Despite its immense efficiency, the framework faces some limitations that set the path for future development. Most notably, its one-chain nature (limited to Ethereum transactions only) makes it blind to advanced, cross-chain fraud schemes that employ inter-chain bridges to obfuscate the source or destination addresses. Second, the batch-processing nature of STGNNs (5 minutes at a time) is prone to missing immediate threats such as flash loan fraud. Lastly, the framework has not been tested for adversarial attacks targeting graph perturbations (intentional evasion techniques). In terms of future directions, developing an HGT-based heterogeneous multi-chain transaction graph is critical for monitoring cross-chain laundering. In the long term, the framework should become resistant to adversarial techniques by being hardened through adversarial training while producing structurally plausible synthetic fraud data with GC-GANs and incorporating GNN fraud scores directly in smart contracts by leveraging oracles (Chainlink).

This will provide pre-execution automated prevention of fraud in the blockchain. Moreover, SAR auto-generation for regulatory compliance is a desirable addition.

References

- [1] Schär, F. (2021). Decentralized Finance: On Blockchain and Smart Contract-Based Financial Markets. *Federal Reserve Bank of St. Louis Review*, 103(2), 153–174.
- [2] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *ICLR 2017*.
- [3] Veličković, P. et al. (2018). Graph Attention Networks. *ICLR 2018*.
- [4] Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *NeurIPS 2017*.
- [5] Vallarino, D. (2025). AI-Powered Fraud Detection in Financial Services: GNN, Compliance Challenges, and Risk Mitigation.
- [6] Al-Harbi, H. (2025). Detecting Anomalies in Blockchain Transactions Using Spatial-Temporal Graph Neural Networks. *AMIT*, 1(1).
- [7] Kadam, P. (2024). Jump-Attentive Graph Neural Network for Financial Fraud Detection. Vesta Corporation.
- [8] Usman, S. O. (2025). Hybrid GCN-GAT Framework for Real-Time DeFi Fraud Detection on Ethereum.
- [9] Ying, Z. et al. (2019). GNNExplainer: Generating Explanations for Graph Neural Networks. *NeurIPS 2019*.
- [10] Luo, B. et al. (2023). AI-Powered Fraud Detection in Decentralized Finance: A Project Life Cycle Perspective. *arXiv:2308.15992*.
- [11] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [12] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum Foundation, 2014.
- [13] M. Bartoletti and L. Pompianu, "An Analysis of Bitcoin OP_RETURN Metadata," in *Financial Cryptography and Data Security*, Springer, 2017, pp. 218-230, doi: 10.1007/978-3-319-70278-0_14.
- [14] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1024-1034.

- [15] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [16] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph Attention Networks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [17] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," in *Proceedings of the KDD 2019 Workshop on Anomaly Detection in Finance*, 2019, pp. 1-13.
- [18] E. Huang, H. Zhong, R. Zhu, and Q. Li, "Ethereum Fraud Detection with Heterogeneous Graph Neural Networks," *IEEE Transactions on Computational Social Systems*, vol. 10, no. 4, pp. 1748-1757, Aug. 2023, doi: 10.1109/TCSS.2022.3181455.
- [19] P. Hu, H. Liu, W. Jiang, and H. Chen, "Transaction-based Hidden Strategies to Compromise an Ethereum DeFi Ecosystem," in *Proc. ACM Conf. on Computer and Communications Security (CCS)*, 2021, pp. 2125-2127, doi: 10.1145/3460120.3485254.
- [20] S. Bartolucci, P. Caccioli, and P. Vivo, "A Percolation Model for the Emergence of the Bitcoin Lightning Network," *Scientific Reports*, vol. 10, no. 1, Art. no. 4488, Mar. 2020, doi: 10.1038/s41598-020-61137-5.
- [21] D. Yoo, K. Kim, S. Kim, and H. Oh, "Detecting Rug Pull Scams in Decentralized Finance Using Machine Learning on Transaction Graphs," in *Proc. IEEE International Conference on Blockchain*, 2022, pp. 284-291, doi: 10.1109/Blockchain55522.2022.00047.
- [22] J. Chen, X. Zheng, Z. Zheng, Y. Wu, and B. Zhou, "Understanding Ethereum via Graph Analysis," *ACM Transactions on Internet Technology*, vol. 20, no. 2, pp. 1-32, May 2020, doi: 10.1145/3381036.
- [23] Y. Li, H. Shen, Z. Rao, and X. Cheng, "Temporal Graph-Based Anomaly Detection in DeFi Protocols," in *Proceedings of The Web Conference (WWW)*, 2023, pp. 3201-3212, doi: 10.1145/3543507.3583395.
- [24] R. Patel and A. Singh, "GNN-Based Phishing Detection on Ethereum Blockchain Using Address Behavior Patterns," *Expert Systems with Applications*, vol. 213, Art. no. 119146, Mar. 2023, doi: 10.1016/j.eswa.2022.119146.

- [25] X. Liu, Z. Dou, and J. Yin, "Towards Understanding Flash Loan Attacks in DeFi: A Transaction Network Perspective," *IEEE Access*, vol. 10, pp. 32845-32858, 2022, doi: 10.1109/ACCESS.2022.3161416.
- [26] K. Toyoda, P. T. Mathiopoulos, I. Sasase, and T. Ohtsuki, "A Novel Blockchain-Based Product Ownership Management System (POMS) for Anti-Counterfeits in the Post Supply Chain," *IEEE Access*, vol. 5, pp. 17465-17477, 2017, doi: 10.1109/ACCESS.2017.2720760.
- [27] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph Neural Networks: A Review of Methods and Applications," *AI Open*, vol. 1, pp. 57-81, 2020, doi: 10.1016/j.aiopen.2021.01.001.
- [28] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A Comprehensive Study on Graph Neural Networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4-24, Jan. 2021, doi: 10.1109/TNNLS.2020.2978386.
- [29] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online Learning of Social Representations," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 701-710, doi: 10.1145/2623330.2623732.
- [30] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 855-864, doi: 10.1145/2939672.2939754.

Contributions



Abhishek Mishra (2207360130004)

Abhishek Mishra served as the team lead and was primarily responsible for designing and implementing the core Graph Neural Network architectures of the project. He architected both the GATv2 (Graph Attention Network v2) and GraphSAGE models using PyTorch Geometric, incorporating multi-head attention mechanisms, batch normalization, dropout regularization, and an Adam optimizer with a ReduceLROnPlateau learning rate scheduler. Beyond the individual GNN models, He engineered the soft-voting ensemble pipeline that integrates GATv2, GraphSAGE, and XGBoost-GraphEmb predictions, averaging class probabilities to produce the final fraud-risk classification. His leadership ensured the project maintained a coherent architecture from graph construction through to final ensemble inference.



Praveen Sharma (2307360139002)

Praveen Sharma was responsible for developing the XGBoost-GraphEmb component, which forms the third pillar of the ensemble. He constructed the 140-dimensional combined feature matrix by concatenating the 64-dimensional GATv2 node embeddings, the 64-dimensional GraphSAGE node embeddings, and the 12-dimensional raw normalised node features. This hybrid representation allowed the gradient-boosted model to exploit both structural graph signals and tabular transaction attributes simultaneously. He configured the XGBClassifier with a multi:softprob objective, 500 estimators, maximum depth of 6, and class-weighted sample balancing to address the severe class imbalance between low-risk, moderate-risk, and high-risk transactions.



Rishabh Bharti (2307360139004)

Rishabh Bharti was responsible for the complete data pipeline, beginning with loading and cleaning the metaverse transaction dataset of 78,600 records. He performed label encoding of categorical columns (transaction type, location region, purchase pattern, age group), extracted temporal features from timestamps (day of week, month), and applied StandardScaler normalisation to produce the final 12-dimensional node feature matrix used as input to both GNN models. He also constructed the directed transaction graph by mapping wallet addresses to unique node identifiers, building the edge index from sender-receiver pairs, and aggregating per-node features via grouped mean over all transactions originating from each address.



Vikas Pratap (2307360139006)

Vikas Pratap designed and executed the complete evaluation framework for the project. He implemented the macro-F1, accuracy, precision, recall, and AUC-ROC (one-vs-rest) metrics for all models, produced per-class ROC and Precision-Recall curves for the ensemble, and generated percentage-normalised confusion matrices for every model including the final ensemble. He compiled the fraud pattern detection analysis (Table 5.2) by mapping test nodes back to transaction types and computing high-risk detection rates per fraud category. As documentation lead, Vikas authored the final project report, ensuring that all experimental results, figures, and tables were accurately described and correctly formatted for submission to Dr. A.P.J. Abdul Kalam Technical University, Lucknow.